

**UNIVERSIDADE CATÓLICA DE BRASÍLIA
PRÓ-REITORIA ACADÊMICA
CURSO DE CIÊNCIAS DA COMPUTAÇÃO
TRABALHO ACADÊMICO**

**FIT UP DOCUMENTO DE VISÃO DO SISTEMA
TESTE DE SOFTWARE**

AUTORES:

**GUSTAVO JUNIO DE SOUZA PEREIRA
RAPHAEL GONDIM OLIVEIRA FILHO
HUGO OLIVEIRA DA SILVA
ISAAC ALONÇO ITACARAMBI DE SOUZA SILVA
PAULO VITOR LUCENA MENDES**

PROFESSOR(A):

SAMUEL NOVAIS MOURA JUNIOR

BRASÍLIA - DF

2026

SUMÁRIO

1. INTRODUÇÃO	4
1.1 Objetivo do Documento	4
1.2 Visão Geral do Produto	4
2. POSICIONAMENTO E PROPOSTA DE VALOR.....	5
3. PERFIS DE USUÁRIO E PARTES INTERESSADAS (STAKEHOLDERS)	5
4. REQUISITOS DO SISTEMA	6
4.1 Requisitos Funcionais (RF)	6
4.2 Requisitos Não-Funcionais (RNF).....	8
5. ARQUITETURA E TECNOLOGIAS EMPREGADAS	9
6. RISCOS DO PRODUTO E CONSIDERAÇÕES DE QUALIDADE	10
6.1 Inconsistência nos Dados de Entrada.....	10
6.2 Concorrência de Acesso ao SQLite	10
6.3 Regressão de Funcionalidades	10
7. VISÃO GERAL DAS HISTÓRIAS DE USUÁRIO.....	11
8. DETALHAMENTO DAS HISTÓRIAS DE USUÁRIO E CENÁRIOS BDD	12
9. IDENTIFICAÇÃO E OBJETIVO DOS TESTES	23
9.1 Objetivos Específicos	23
10. ESCOPO DOS TESTES	24
10.1 Escopo dos Testes	24
10.2 Módulo de Validações e Regras de Negócio (Camada Unitária):	24
10.3 Módulo de Serviços e API REST (Camada de Integração):	24
10.4 Módulo de Interface e Interação do Usuário (Camada E2E):	25
10.5 Testes de Regressão.....	25
10.6 Itens Fora do Escopo (Out-of-Scope)	25
11. FERRAMENTAS E TECNOLOGIAS DE TESTE	26
12. ESTRATÉGIA DE TESTES	27
12.1 Pirâmide de Testes.....	27
12.2 Técnicas de Concepção de Teste	28
12.3 Critérios de Aceitação e Parada	29
13. AMBIENTE DE TESTES.....	30
13.1 Configuração de Hardware e Software	30
14. PAPÉIS E RESPONSABILIDADES DA EQUIPE (GRUPO DE ATÉ 5 ALUNOS).....	31
15. GERENCIAMENTO DE RISCOS NO PROCESSO DE TESTES.....	33
15.1 Sumário dos Casos de Teste	34
16. ESPECIFICAÇÃO DOS TESTES UNITÁRIOS.....	36
17. ESPECIFICAÇÃO DOS TESTES DE API (ENDPOINTS REST).....	44

18. ESPECIFICAÇÃO DOS TESTES E2E (INTERFACE GRÁFICA WEB)	54
19. MATRIZ DE EXECUÇÃO E RASTREABILIDADE DE TESTES	60
20. LOGS DE EXECUÇÃO DAS FERRAMENTAS	62
20.1 Evidência 1: Log de Testes Unitários (Maven + JUnit 5)	62
20.2 Evidência 2: Log de Testes de API (Newman / Postman CLI)	63
20.3 Evidência 3: Log de Testes E2E (Playwright)	64
21. RESULTADOS DA COBERTURA E DO GERENCIAMENTO DE DEFEITOS	65
21.1 Relatório de Cobertura de Código (JaCoCo)	65
21.2 Registro Formal de Defeitos e Bugs Encontrados (Bug Tracking)	66
22. INTRODUÇÃO AO SISTEMA DESENVOLVIDO	69
22.1 Contextualização e Propósito do Sistema FIT UP	69
22.2 Arquitetura e Tecnologias Utilizadas	69
22.3 Módulos e Funcionalidades Principais	70
22.4 Documentação de Requisitos e Cenários BDD (Gherkin)	71
23. PLANEJAMENTO DE TESTES	72
23.1 Objetivo dos Testes	72
23.2 Escopo dos Testes	72
23.3 Ferramentas e Frameworks Utilizados	72
23.4 Estratégia de Testes e Técnicas de Caixa-Preta	73
23.5 Critérios de Aceitação e Parada	73
23.6 Papéis e Responsabilidades no Grupo (Matriz RACI)	74
24. DESCRIÇÃO DOS CASOS DE TESTE	74
24.1 Testes Unitários (Validações de Regras e Classes Individuais)	74
24.2 Testes de API (Endpoints REST)	76
24.3 Testes Ponta a Ponta (E2E)	77
25. RESULTADOS DAS EXECUÇÕES E EVIDÊNCIAS	78
25.1 Matriz de Execução e Status Consolidado	78
25.2 Evidências e Logs de Execução	79
25.3 Métricas de Cobertura de Código (JaCoCo)	81
25.4 Identificação e Relatório de Defeitos (Bug Tracking)	81
26. CONCLUSÃO E LIÇÕES APRENDIDAS	82
26.1 Lições Aprendidas pela Equipe	82
26.2 Lições Aprendidas pela Equipe	82
26.3 Trabalhos Futuros	83
27. LINK PARA O REPOSITÓRIO NO GITHUB	83
Equipe / Grupo de Trabalho	83

1. INTRODUÇÃO

1.1 Objetivo do Documento

Este Documento de Visão tem como finalidade estabelecer o escopo, os objetivos de negócio, o público-alvo, as necessidades dos usuários e os requisitos funcionais e não funcionais do sistema **FIT UP**.

O documento servirá como base orientadora para todo o ciclo de vida do desenvolvimento do sistema, além de auxiliar no planejamento, na execução e no acompanhamento das atividades de Garantia da Qualidade (QA) e dos testes de software.

1.2 Visão Geral do Produto

O **FIT UP** é uma solução destinada à gestão operacional e financeira de academias e centros de treinamento físico. O sistema centraliza o gerenciamento de alunos, instrutores, planos de assinatura, matrículas e fluxo de pagamentos.

A solução foi desenvolvida utilizando uma arquitetura híbrida, com foco em desempenho e independência de infraestrutura externa. O backend é desenvolvido em **Java**, utilizando os princípios de **Programação Orientada a Objetos (POO)**, enquanto o banco de dados **SQLite** é incorporado à aplicação, eliminando a necessidade de servidores externos para seu funcionamento.

O sistema também conta com um **servidor HTTP com API REST integrada** e oferece duas formas de interação com o usuário: uma **interface web moderna**, baseada no modelo **SPA (Single Page Application)**, e uma **interface de linha de comando (CLI)**.

2. POSICIONAMENTO E PROPOSTA DE VALOR

ELEMENTO	DESCRIÇÃO
Para	Gestores de academia, recepcionistas e instrutores de condicionamento físico
Que	Necessitam de uma ferramenta rápida, confiável e sem dependências complexas de infraestrutura
O FIT UP é	Um sistema integrado de gestão operacional, clínica e financeira para academias
Que proporciona	Cadastro unificado de alunos e instrutores, cálculo automático de IMC, controle de mensalidades e métricas de desempenho em tempo real
Diferencial	Versão 100% portátil com banco de dados SQLite local, servidor embutido e operação tanto via navegador quanto via terminal CLI

3. PERFIS DE USUÁRIO E PARTES INTERESSADAS (STAKEHOLDERS)

PAPEL	DESCRIÇÃO	RESPONSABILIDADES NO SISTEMA
Administrador / Gestor	Responsável pela gestão geral da academia	Cadastrar planos, monitorar receitas, emitir relatórios, gerenciar acessos e visualizar dashboards globais.
Recepcionista / Atendente	Operador diário da recepção da academia	Cadastrar alunos, vincular planos, efetuar recebimento de mensalidades e atualizar status de inadimplência.
Instrutor / Personal	Profissional de educação física	Consultar prontuário de alunos, acompanhar métricas biométricas (peso, altura, IMC) e evolução dos treinos.
Equipe de QA / Testes	Responsáveis pela qualidade do software	Elaborar planos de teste, especificar casos de teste (Unitário, API e E2E), executar baterias de validação e relatar defeitos.

4. REQUISITOS DO SISTEMA

4.1 Requisitos Funcionais (RF)

ID	NOME DO REQUISITO	DESCRIÇÃO
RF01	Autenticação e controle de acesso	O sistema deve permitir que usuários autorizados realizem login com credenciais válidas (e-mail e senha) para acessar o painel administrativo.
RF02	Gestão de alunos (CRUD)	O sistema deve permitir que cadastrar, listar, detalhar, atualizar e inativar/remover registros de alunos, incluindo dados cadastrais e biométricos.
RF03	Cálculo automático de IMC	O sistema deve calcular automaticamente o Índice de Massa Corporal (IMC) do aluno ($IMC = \frac{\text{peso}}{\text{altura}^2}$) e categorizá-lo (Abaixo do peso, Peso normal, Sobrepeso, Obesidade).
RF04	Gestão de instrutores (CRUD)	O sistema deve permitir cadastrar, listar, editar e excluir instrutores, registrando nome, CPF, e-mail, telefone e especialidade esportiva.
RF05	Gestão de planos de assinatura (CRUD)	O sistema deve permitir cadastrar, editar e excluir planos da academia com seus respectivos valores mensais em reais.
RF06	Matrícula e vinculação de aluno e plano	O sistema deve permitir vincular um aluno a um plano ativo no momento do cadastro ou posterior alteração.

ID	NOME DO REQUISITO	DESCRIÇÃO
RF07	Gestão financeira e pagamentos (CRUD)	O sistema deve permitir registrar pagamentos associados a alunos e planos, com controle de status (Pago, Pendente, Atrasado).
RF08	Painel de indicadores (Dashboard)	O sistema deve disponibilizar um painel gerencial consolidado com contadores em tempo real de alunos ativos, instrutores, planos e total de receitas.
RF09	Dupla interface de operação	O sistema deve ser operacional tanto através de navegador Web (HTML5/CSS3/JavaScript) quanto via console de linha de comando (CLI interativo).
RF10	Validação rígida de dados de entrada	O sistema deve validar formato de CPF (11 dígitos), formato de e-mail, integridade de datas (sem datas futuras de nascimento), limites biométricos (altura positiva até 3m) e valores financeiros positivos.

4.2 Requisitos Não-Funcionais (RNF)

ID	NOME DO REQUISITO	DESCRIÇÃO
RNF01	Portabilidade	O sistema não deve exigir instalação de servidores de aplicação externos (como Tomcat) ou SGBDs dedicados; deve operar via arquivo executável JAR único com SQLite embutido.
RNF02	Desempenho	O tempo de resposta para qualquer requisição HTTP aos endpoints da API REST local não deve exceder 300 ms em condições normais de uso.
RNF03	Compatibilidade	A interface Web deve ser compatível com os principais navegadores modernos (Google Chrome, Mozilla Firefox, Microsoft Edge e Safari) com design responsivo.
RNF04	Usabilidade	A interface gráfica deve adotar padrão estético moderno com tema escuro, efeitos de glassmorphism, feedback visual imediato via alertas flutuantes (toasts) e tipografia padronizada (Inter).
RNF05	Manutenibilidade e POO	O backend deve ser desenvolvido aplicando rigorosamente os conceitos de Orientação a Objetos (Encapsulamento, Herança, Polimorfismo e Abstração) e separação de camadas.
RNF06	Testabilidade	O código-fonte deve ser desacoplado para viabilizar a escrita de testes automatizados em três níveis: Unitário, Integração de API e Ponta a Ponta (E2E).

5. ARQUITETURA E TECNOLOGIAS EMPREGADAS

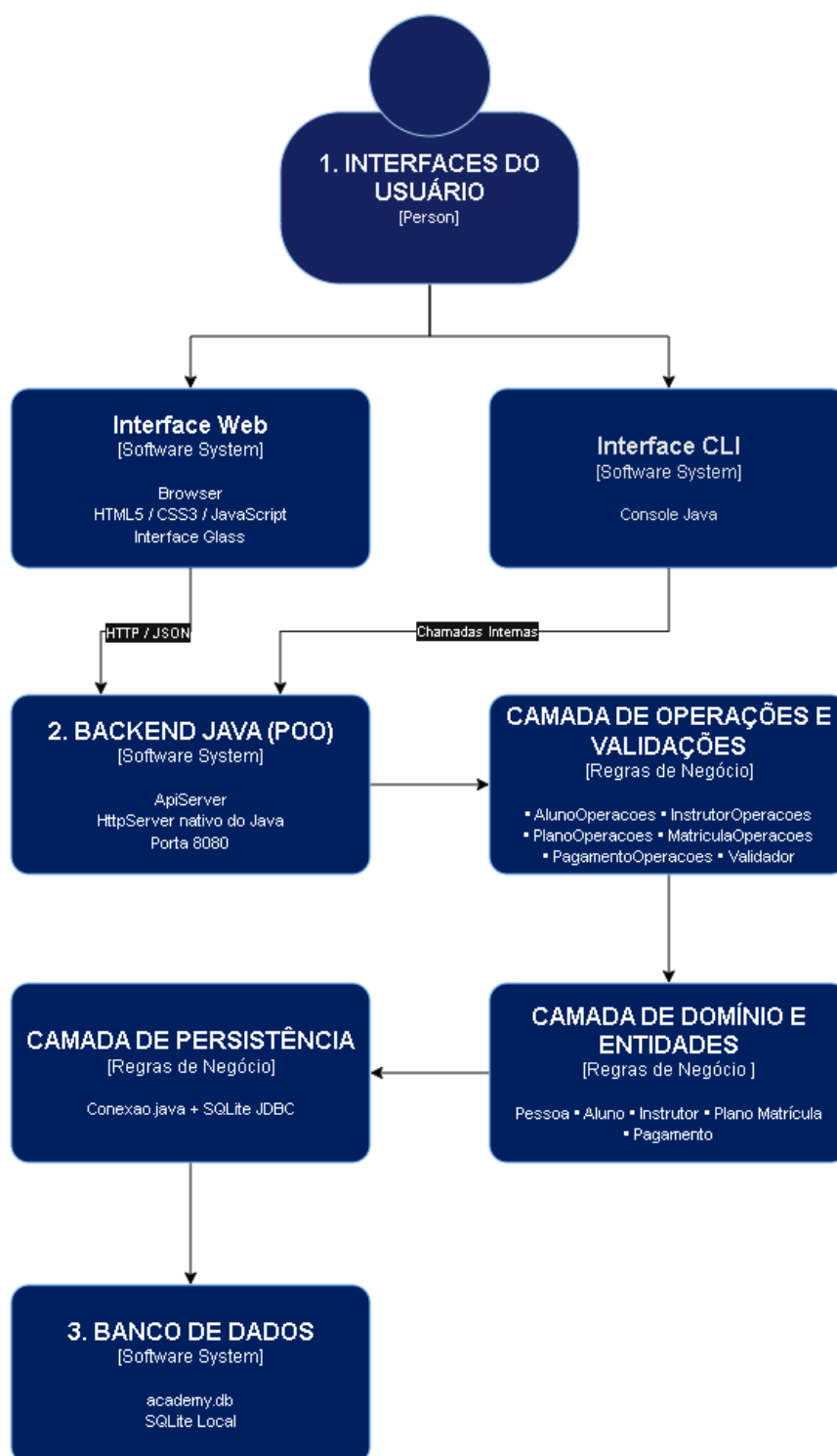


Figura 1 – Diagrama de Arquitetura de Software

Fonte: Elaborado pelos autores (2026).

6. RISCOS DO PRODUTO E CONSIDERAÇÕES DE QUALIDADE

6.1 Inconsistência nos Dados de Entrada

Risco: Inserção de dados inválidos ou malformados, como CPFs em formato incorreto, idades negativas ou valores nulos.

Mitigação: Implementação de validações em duas camadas: no frontend, por meio de máscaras e recursos de validação do HTML5; e no backend, por meio da classe `Validador.java`. A eficácia das validações será verificada por meio de testes unitários.

6.2 Concorrência de Acesso ao SQLite

Risco: O SQLite pode apresentar bloqueios do arquivo durante operações de escrita simultâneas, podendo ocasionar falhas ou conflitos no acesso aos dados.

Mitigação: Realização de testes de integração da API para validar o comportamento das transações, garantir a atomicidade das operações e verificar o correto tratamento das exceções relacionadas ao banco de dados.

6.3 Regressão de Funcionalidades

Risco: Alterações nas regras de negócio, como o cálculo do IMC ou a definição do status de pagamento, podem afetar funcionalidades que já estavam funcionando corretamente.

Mitigação: Desenvolvimento e manutenção de uma suíte automatizada de testes unitários e de API, integrada ao ciclo de desenvolvimento e entrega do sistema.

7. VISÃO GERAL DAS HISTÓRIAS DE USUÁRIO

CÓDIGO	TÍTULO DA HISTÓRIA DE USUÁRIO	PRIORIDADE	MÓDULO
US01	Autenticação no Sistema (Login de Operador)	Alta (Obrigatório)	Segurança /Acesso
US02	Cadastro e Gestão de Alunos com Biometria	Alta (Obrigatório)	Cadastros/Alunos
US03	Classificação Automática de IMC	Média (Should Have)	Regra de Negócio
US04	Cadastro e Gestão de Planos de Mensalidade	Alta (Obrigatório)	Financeiro/Planos
US05	Matrícula e Associação de Aluno a Plano	Alta (Obrigatório)	Operacional
US06	Registro e Gestão de Status de Pagamentos	Alta (Obrigatório)	Financeiro/Pagamentos
US07	Gestão do Corpo de Instrutores	Média (Should Have)	Cadastros/Instrutores
US08	Visualização de Indicadores no Dashboard	Baixa (Could Have)	Gerencial/Relatórios

8. DETALHAMENTO DAS HISTÓRIAS DE USUÁRIO E CENÁRIOS BDD

US01: Autenticação no Sistema (Login do Operador)

Como atendente ou administrador da academia, **eu quero** me autenticar informando meu e-mail e senha cadastrados, **para que** eu possa acessar com segurança as funcionalidades de gestão do FIT UP.

Critérios de Aceitação

1. O sistema deve permitir que o usuário informe seu e-mail e senha.
2. Quando as credenciais forem válidas, o sistema deve liberar o acesso ao painel principal e
3. estabelecer uma sessão autenticada ou disponibilizar um token de autenticação válido.
4. Quando o e-mail não estiver cadastrado ou a senha estiver incorreta, o sistema deve exibir a mensagem: *"Credenciais inválidas. Verifique seu e-mail e senha."*
5. O sistema não deve permitir o envio do formulário quando os campos de e-mail ou senha estiverem vazios.

Cenários em BDD (Gherkin)

Funcionalidade: Autenticação de Operador

Como um atendente da academia

Quero fazer login no sistema FIT UP

Para acessar o painel de gestão de alunos e pagamentos

Cenário: Login efetuado com sucesso (Caminho Feliz)

Dado que o operador está na tela de login do FIT UP

E que existe um operador cadastrado com e-mail "admin@fitup.com" e senha "Senha@123"

Quando ele preencher o campo "E-mail" com "admin@fitup.com"

E preencher o campo "Senha" com "Senha@123"

E clicar no botão "Entrar"

Então o sistema deve autenticar o operador com sucesso

E redirecionar o operador para a página do Dashboard

E exibir a mensagem "Login realizado com sucesso!"

Cenário: Tentativa de login com senha incorreta (Fluxo de Exceção)

Dado que o operador está na tela de login do FIT UP

E que existe um operador cadastrado com e-mail "admin@fitup.com"

Quando ele preencher o campo "E-mail" com "admin@fitup.com"

E preencher o campo "Senha" com "SenhaErrada999"

E clicar no botão "Entrar"

Então o sistema deve recusar a autenticação

E permanecer na tela de login

E exibir o alerta de erro "Credenciais inválidas. Verifique seu e-mail e senha."

Cenário: Tentativa de login com campos vazios (Fluxo Alternativo)

Dado que o operador está na tela de login do FIT UP

Quando ele deixar os campos de e-mail e senha vazios

E tentar submeter o formulário

Então o sistema deve destacar os campos obrigatórios

E impedir o envio da requisição de autenticação

US02: Cadastro e Gestão de Alunos com Dados Biométricos

Como recepcionista da academia, **eu quero** cadastrar um novo aluno informando seus dados cadastrais e medidas físicas, **para que** o aluno seja registrado no banco de dados e possa frequentar os treinos.

Critérios de Aceitação

1. Os campos nome, CPF, e-mail e data de nascimento devem ser de preenchimento obrigatório.
2. O nome do aluno deve possuir, no mínimo, 3 caracteres.
3. O CPF deve conter exatamente 11 dígitos numéricos e ser válido.
4. O e-mail deve possuir uma estrutura sintática válida, no formato `usuario@dominio.ext`.
5. A data de nascimento não pode ser posterior à data atual.
6. A altura deve ser informada em metros, com valor maior que zero e menor ou igual a 3,0 m.
7. O peso deve ser informado em quilogramas, com valor maior ou igual a zero.

Cenários em BDD (Gherkin):

Funcionalidade: Cadastro de Alunos

Como recepcionista

Quero registrar novos alunos

Para manter o controle dos clientes da academia

Cenário: Cadastro de aluno realizado com sucesso (Caminho Feliz)

Dado que o recepcionista está na aba "Alunos"

E aciona o botão "Novo Aluno"

Quando preenche o formulário com:

E clica em "Salvar Aluno"

Então o sistema deve persistir os dados do aluno no banco de dados SQLite

E fechar a janela modal

E exibir a notificação de sucesso "Aluno salvo com sucesso!"

E listar "Lucas Mendes Ferreira" na tabela de alunos

CAMPO	VALOR
Nome Completo	"Lucas Mendes Ferreira"
CPF	"123.456.789-00"
E-mail	"lucas.mendes@fit.com"
Telefone	"(11) 98765-4321"
Endereço	"Av. Paulista, 1000"
Data de Nascimento	"15/05/1995"
Peso	"78.5"
Altura	"1.80"

Cenário: Tentativa de cadastro com CPF inválido (Fluxo de Exceção)

Dado que o recepcionista está preenchendo o formulário de aluno

Quando informa o CPF "12345", contendo menos de 11 dígitos

E clica em "Salvar Aluno"

Então o sistema deve bloquear o cadastro

E exibir a mensagem de erro "CPF inválido. Certifique-se de que possui 11 dígitos."

E manter as demais informações preenchidas no formulário

Cenário: Tentativa de cadastro com data de nascimento no futuro (Fluxo de Exceção)

Dado que o recepcionista está preenchendo o formulário de aluno

Quando informa a data de nascimento "31/12/2099"

E clica em "Salvar Aluno"

Então o sistema deve bloquear o cadastro

E exibir a mensagem de erro "Data de nascimento inválida (não pode ser no futuro)."

E manter as informações preenchidas no formulário

US03: Classificação Automática de IMC (Índice de Massa Corporal)

Como instrutor de academia, **eu quero** que o sistema calcule automaticamente o IMC do aluno a partir do peso e da altura, **para que** eu possa consultar o resultado do cálculo e utilizar essa informação como apoio na avaliação do aluno e na definição do treino.

Critérios de Aceitação

1. O sistema deve calcular o IMC utilizando a fórmula: $IMC = \text{peso} / \text{altura}^2$
2. Para valores de IMC menores que 18,5, o sistema deve apresentar a classificação "**Abaixo do peso**".
3. Para valores de IMC maiores ou iguais a 18,5 e menores que 25,0, o sistema deve apresentar a classificação "**Peso normal**".
4. Para valores de IMC maiores ou iguais a 25,0 e menores que 30,0, o sistema deve apresentar a classificação "**Sobrepeso**".
5. Para valores de IMC maiores ou iguais a 30,0, o sistema deve apresentar a classificação "**Obesidade**".
6. Caso a altura seja igual a zero ou não esteja informada, o sistema não deve realizar o cálculo e deve apresentar a classificação "**Não calculado**", evitando uma divisão por zero.

Cenários em BDD (Gherkin):

Funcionalidade: Cálculo e Classificação de IMC

Como instrutor

Quero consultar a classificação do IMC dos alunos

Para utilizar os dados biométricos como apoio na avaliação dos alunos

Esquema do Cenário: Validação das faixas de IMC

Dado que um aluno possui peso de kg e altura de m

Quando o sistema efetua o cálculo do IMC

Então o valor do IMC calculado deve ser aproximadamente <imc_esperado>

E a classificação retornada deve ser ""

Exemplos:

PESO	ALTURA	IMC ESPERADO	CLASSIFICAÇÃO
50.0	1.75	16.3	"Abaixo do peso"
70.0	1.75	22.9	"Peso normal"
80.0	1.70	27.7	"Sobrepeso"
105.0	1.75	34.3	"Obesidade"

Cenário: IMC não calculado por altura igual a zero

Dado que um aluno possui peso de 70.0 kg e altura de 0.0 m

Quando o sistema tenta efetuar o cálculo do IMC

Então o sistema não deve realizar a divisão

E deve retornar a classificação "Não calculado"

E não deve ocorrer erro de divisão por zero

US04: Cadastro e Gestão de Planos de Mensalidade

Como gestor da academia, **eu quero** cadastrar planos com nomes e valores mensais em reais, **para que** a academia ofereça opções contratuais claras aos alunos.

Critérios de Aceitação

1. O nome do plano não pode ser nulo, vazio ou conter apenas espaços em branco.
2. O valor mensal do plano deve ser estritamente maior que zero ($R\$ > 0,00$).
3. Após o cadastro, o plano deve ser listado imediatamente na tabela de planos e disponibilizado para seleção no formulário de matrícula.

Cenários em BDD (Gherkin):

Funcionalidade: Gestão de Planos

Como gestor

Quero cadastrar novos planos de assinatura

Para disponibilizar opções de mensalidade aos alunos

Cenário: Cadastro de novo plano com sucesso (Caminho Feliz)

Dado que o gestor está na tela de "Planos"

E clica em "Novo Plano"

Quando preenche o nome com "Plano Gold Anual"

E preenche o valor mensal com "129.90"

E clica em "Salvar Plano"

Então o plano deve ser persistido no banco de dados SQLite

E exibido na listagem com nome "Plano Gold Anual" e valor "R\$ 129,90"

E o sistema deve exibir a mensagem "Plano salvo com sucesso!"

E o plano deve estar disponível para seleção no formulário de matrícula

Cenário: Tentativa de cadastro de plano com valor zero ou negativo (Fluxo de Exceção)

Dado que o gestor está no formulário de criação de plano

Quando preenche o nome com "Plano Grátis Promocional"

E preenche o valor mensal com "-10.00"

E clica em "Salvar Plano"

Então o sistema deve rejeitar o cadastro

E exibir a mensagem de validação "O valor do plano deve ser maior que zero."

Cenário: Tentativa de cadastro de plano sem nome (Fluxo de Exceção)

Dado que o gestor está no formulário de criação de plano

Quando deixa o campo "Nome" vazio

E preenche o valor mensal com "99.90"

E clica em "Salvar Plano"

Então o sistema deve rejeitar o cadastro

E exibir uma mensagem informando que o nome do plano é obrigatório

E não deve persistir o plano no banco de dados

US05: Matrícula e Associação de Aluno a Plano

Como recepcionista, **eu quero** associar um aluno a um plano ativo no momento do cadastro ou na edição, **para que** seja gerado o vínculo de matrícula da academia.

Critérios de Aceitação:

1. O recepcionista pode selecionar um plano existente na caixa de seleção (<select>).
2. Se selecionado, o sistema cria/atualiza a matrícula correspondente na tabela **mátrículas**.
3. Caso a opção "Sem plano" seja escolhida, o aluno é salvo sem vínculo de matrícula ativo.

Cenários em BDD (Gherkin):

Funcionalidade: Associação de Aluno a Plano

Como recepcionista

Quero matricular o aluno em um plano contratado

Para formalizar o vínculo contratual

Cenário: Matrícula de aluno em plano existente (Caminho Feliz)

Dado que existem os planos "Plano Básico" (R\$ 89,90) e "Plano Black" (R\$ 149,90) cadastrados

E o recepcionista está cadastrando o aluno "Mariana Rios"

Quando ele seleciona o plano "Plano Black" no campo de plano

E submete o formulário

Então o sistema deve gravar o aluno

E criar um registro de matrícula associando o ID do aluno ao ID do "Plano Black"

E exibir o nome do plano "Plano Black" na linha da tabela de alunos

Cenário: Alteração do plano de um aluno matriculado

Dado que o aluno "Mariana Rios" está matriculado no "Plano Básico"

E o recepcionista está editando o cadastro do aluno

Quando ele seleciona o plano "Plano Black" no campo de plano

E submete o formulário

Então o sistema deve atualizar o registro de matrícula correspondente na tabela matrículas

E associar o aluno ao "Plano Black"

E exibir o nome do novo plano na linha da tabela de alunos

Cenário: Cadastro de aluno sem plano

Dado que o recepcionista está cadastrando o aluno "Mariana Rios"

Quando ele seleciona a opção "Sem plano" no campo de plano

E submete o formulário

Então o sistema deve gravar o aluno

E não deve criar um vínculo de matrícula ativo

E o aluno deve permanecer sem plano associado

US06: Registro e Gestão de Pagamentos de Mensalidade

Como recepcionista ou gestor financeiro, **eu quero** registrar pagamentos de mensalidades e definir seus respectivos status (Pago, Pendente ou Atrasado), **para que** a academia mantenha o controle do fluxo de caixa e possa identificar situações de inadimplência.

Critérios de Aceitação

1. O sistema deve exigir a seleção de um aluno válido e previamente cadastrado para registrar um pagamento.
2. O valor do pagamento não pode ser negativo.
3. O status do pagamento deve aceitar exclusivamente os valores "**Pago**", "**Pendente**" ou "**Atrasado**".
4. Após o registro, o pagamento deve ser armazenado no banco de dados e associado corretamente ao aluno selecionado.

Cenários em BDD (Gherkin)

Funcionalidade: Registro de Pagamento

Como operador financeiro

Quero lançar os pagamentos recebidos dos alunos

Para manter o fluxo de caixa atualizado

Cenário: Lançamento de pagamento efetuado (Caminho Feliz)

Dado que o aluno "Lucas Mendes" possui mensalidade no valor de R\$ 129,90

E o operador acessa a seção "Pagamentos"

E clica em "Novo Pagamento"

Quando ele seleciona o aluno "Lucas Mendes"

E o sistema preenche automaticamente o valor "129.90"

E o operador define o status como "Pago"

E confirma o lançamento

Então o sistema deve registrar a transação com a data atual

E associar o pagamento ao aluno selecionado

E atualizar o contador de receitas no Dashboard

E exibir o registro na tabela de pagamentos com o status "Pago"

Cenário: Tentativa de registrar pagamento sem selecionar aluno (Fluxo de Exceção)

Dado que o operador abre o modal de pagamento

Quando ele não seleciona nenhum aluno

E clica em "Registrar Pagamento"

Então o sistema deve bloquear o envio

E apresentar o alerta "Um aluno válido deve ser selecionado para o pagamento."

E não deve registrar o pagamento no banco de dados

Cenário: Tentativa de registrar pagamento com valor negativo (Fluxo de Exceção)

Dado que o operador está no formulário de registro de pagamento

E o aluno "Lucas Mendes" está selecionado

Quando ele informa o valor "-50.00"

E define o status como "Pago"

E clica em "Registrar Pagamento"

Então o sistema deve bloquear o registro

E exibir a mensagem "O valor do pagamento não pode ser negativo."

E não deve persistir o pagamento no banco de dados

US07: Cadastro e Gestão de Instrutores

Como gestor da academia, **eu quero** cadastrar instrutores informando suas áreas de especialidade (Musculação, Pilates, Crossfit ou Lutas), **para que** os alunos possam conhecer a equipe técnica e agendar treinos específicos.

Critérios de Aceitação:

1. Nome, CPF e Especialidade são campos obrigatórios.
2. O nome do instrutor deve possuir no mínimo 3 caracteres.
3. O CPF deve conter exatamente 11 dígitos válidos.
4. A especialidade deve ser informada e não pode estar vazia.

Funcionalidade: Gestão de Instrutores

Como gestor da academia quero manter o cadastro da equipe de instrutores
Para alocar os profissionais nas modalidades de treino

Cenário: Cadastro de instrutor com especialidade (Caminho Feliz)

Dado que o gestor está na aba "Instrutores"

E clica em "Novo Instrutor"

Quando preenche os campos com:

E clica em "Salvar Instrutor"

Então o instrutor deve ser gravado no banco SQLite

E a tabela de instrutores deve exibir uma nova linha com o nome, CPF e especialidade informados.

CAMPO	VALOR
Nome Completo	"Rodrigo Silva Santos"
CPF	"987.654.321-11"
E-mail	"rodrigo.coach@fit.com"
Especialidade	"Musculação"

Cenário: Cadastro com nome inválido (Fluxo de Exceção)

Dado que o gestor está na aba "Instrutores"

E clica em "Novo Instrutor"

Quando informa um nome com menos de 3 caracteres

E preenche os demais campos corretamente

E clica em "Salvar Instrutor"

Então o sistema deve impedir o cadastro

E informar que o nome deve possuir pelo menos 3 caracteres

E o instrutor não deve ser gravado no banco SQLite.

Cenário: Cadastro com CPF inválido (Fluxo de Exceção)

Dado que o gestor está na aba "Instrutores"

E clica em "Novo Instrutor"

Quando informa um CPF com quantidade de dígitos inválida

E preenche os demais campos corretamente

E clica em "Salvar Instrutor"

Então o sistema deve impedir o cadastro

E informar que o CPF é inválido

E o instrutor não deve ser gravado no banco SQLite.

Cenário: Cadastro sem especialidade (Fluxo Alternativo)

Dado que o gestor está na aba "Instrutores"

E clica em "Novo Instrutor"

Quando deixa o campo "Especialidade" vazio

E preenche os demais campos corretamente

E clica em "Salvar Instrutor"

Então o sistema deve impedir o cadastro

E informar que a especialidade é obrigatória

E o instrutor não deve ser gravado no banco SQLite.

9. IDENTIFICAÇÃO E OBJETIVO DOS TESTES

O objetivo deste Plano de Testes é definir a estratégia, o escopo, a arquitetura de ferramentas, os critérios de aceitação e as responsabilidades relacionados à validação da qualidade do sistema **FIT UP**.

O plano tem como finalidade assegurar que o software atenda aos requisitos funcionais, incluindo regras de negócio, operações de CRUD, cálculos biométricos de IMC e fluxos financeiros, bem como aos requisitos não funcionais relacionados ao desempenho, à usabilidade, à estabilidade e à integridade dos dados armazenados no SQLite.

Dessa forma, busca-se identificar e prevenir defeitos durante o ciclo de desenvolvimento, reduzindo os riscos de falhas e contribuindo para a entrega de um sistema confiável e adequado ao ambiente de produção.

9.1 Objetivos Específicos

- **Garantir a confiabilidade das regras de negócio** por meio de **testes unitários**, abrangendo as validações de entrada implementadas na classe **Validador.java** e os cálculos biométricos realizados pela classe **Aluno.java**.
- **Validar a robustez dos endpoints HTTP do backend** por meio de **testes de API**, verificando códigos de status HTTP (200, 201, 400, 404 e 500), headers e payloads em formato JSON.
- **Avaliar a experiência do usuário e a integridade dos fluxos ponta a ponta** por meio de **testes E2E (End-to-End)**, simulando a navegação e as operações realizadas por atendentes e gestores no navegador.
- **Documentar evidências auditáveis das execuções dos testes** e estabelecer um fluxo estruturado para identificação, registro, acompanhamento e correção de defeitos.

10. ESCOPO DOS TESTES

10.1 Escopo dos Testes

Os seguintes módulos, camadas e funcionalidades serão submetidos a testes sistemáticos:

10.2 Módulo de Validações e Regras de Negócio (Camada Unitária):

- Validação de CPF, incluindo quantidade de dígitos, formatação e rejeição de valores inválidos.
- Validação do formato de e-mail, verificando a estrutura esperada para endereços eletrônicos.
- Validação de datas, incluindo a proibição de datas de nascimento futuras.
- Validação de dados biométricos, considerando altura entre 0,01 m e 3,00 m e peso não negativo.
- Cálculo do IMC e classificação de acordo com as quatro faixas definidas nas regras de negócio.
- Normalização de dados, incluindo remoção de espaços em branco com `trim ()`, padronização de telefones e normalização de status.

10.3 Módulo de Serviços e API REST (Camada de Integração):

- Endpoints `/api/alunos`: operações GET, POST, PUT e DELETE.
- Endpoints `/api/instrutores`: operações GET, POST, PUT e DELETE.
- Endpoints `/api/planos`: operações GET, POST, PUT e DELETE.
- Endpoints `/api/mátrículas`: operações GET e POST.
- Endpoints `/api/pagamentos`: operações GET, POST, PUT e DELETE.
- Módulo de autenticação/login: endpoint POST `/api/login`.
- Validação dos cabeçalhos CORS e do `Content-Type: application/json`.
- Validação dos códigos de resposta HTTP e dos payloads retornados pela API.

10.4 Módulo de Interface e Interação do Usuário (Camada E2E):

- Fluxo completo de autenticação, incluindo bloqueio de credenciais incorretas.
- Fluxo de cadastro de um plano de mensalidade e sua exibição na listagem.
- Fluxo de cadastro de um novo aluno, cálculo dinâmico do IMC e associação ao plano cadastrado.
- Fluxo de lançamento de pagamento, alteração de status e atualização dos cards estatísticos do Dashboard.
- Abertura e fechamento de modais e exibição de mensagens de feedback ao usuário, incluindo notificações do tipo toast.

10.5 Testes de Regressão

- Reexecução das suítes de testes automatizados após a implementação de novas funcionalidades ou alterações relevantes no sistema, visando identificar possíveis impactos em funcionalidades existentes.

10.6 Itens Fora do Escopo (Out-of-Scope)

- Testes de carga em escala superior a 5.000 requisições simultâneas por segundo, considerando a proposta de utilização local do sistema e o uso do SQLite.
- Testes avançados de segurança e testes de penetração (PenTest), incluindo técnicas especializadas de exploração de infraestrutura, sistema operacional ou kernel.
- Testes em navegadores descontinuados, como o Internet Explorer 11 ou versões anteriores.
- Testes de internacionalização e suporte a múltiplos idiomas, uma vez que o sistema foi projetado para utilização em português do Brasil (pt-BR).

11. FERRAMENTAS E TECNOLOGIAS DE TESTE

NIVEL/FINALIDADE	FERRAMENTA /FRAMEWORK	VERSÃO	JUSTIFICATIVA TÉCNICA
Testes Unitários	JUnit 5 (Jupiter)	5.10.x	Padrão da indústria para aplicações Java; suporte nativo a testes parametrizados (@ParameterizedTest), asserções fluentes e anotações de ciclo de vida.
Asserções e Mocking	Assert e Mockito	3.25.x/5.11.x	Proporcionam asserções expressivas e legíveis (assertThat(...)) e isolamento de dependências de banco de dados quando necessário.
Testes de API	Postman/Newman e REST-Assured	Postman v10+/REST-Assured 5.4.x	Criação visual de coleções de requisições, validação de JSON Schema e execução automatizada via linha de comando (CLI) pelo Newman.
Testes E2E (Ponta a Ponta)	Playwright/Cypress	1.42.x	Automação rápida e estável de navegadores modernos (Chromium, Firefox, WebKit), com suporte a esperas automáticas (auto-waiting) e captura de screenshots de falhas.
Cobertura de Código	JaCoCo (Cobertura de Código Java)	811	Medição objetiva da cobertura de linhas, branches e instruções no backend Java.
Controle de Versão	Git e GitHub	-	Armazenamento do código, histórico de commits, gestão de branches e publicação dos relatórios.

12. ESTRATÉGIA DE TESTES

12.1 Pirâmide de Testes

A estratégia de testes do **FIT UP** será baseada na **Pirâmide de Testes**, conceito associado a Mike Cohn, priorizando uma maior quantidade de testes rápidos, automatizados e de menor custo de execução, complementados por testes de integração e testes de ponta a ponta.

1. 70% — Testes Unitários:

Foco nas regras de negócio e componentes isolados do sistema, abrangendo validações de dados, tratamento de strings, conversões de tipos e cálculos matemáticos.

2. 20% — Testes de API / Integração:

Validação da camada de controle HTTP, incluindo o processamento dos endpoints REST, a serialização e desserialização de dados utilizando Gson e a comunicação com o banco de dados SQLite.

3. 10% — Testes E2E (End-to-End):

Validação dos principais fluxos de negócio de ponta a ponta por meio da interface web, simulando as ações dos usuários e verificando a integração entre interface, backend e banco de dados.

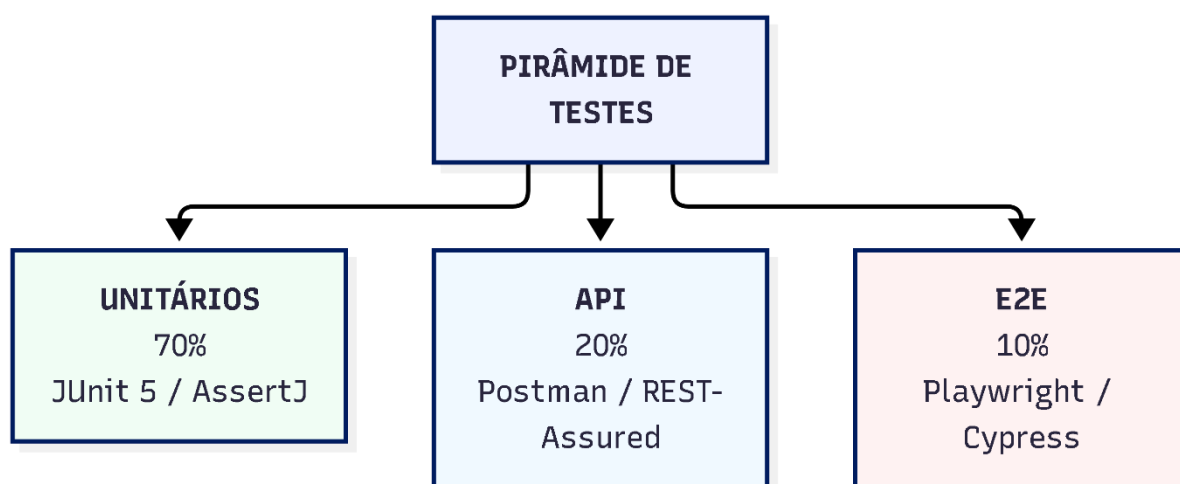


Figura 2 – Pirâmide de Testes

Fonte: Elaborado pelos autores (2026).

12.2 Técnicas de Concepção de Teste

- **Partição de Equivalência (Equivalence Partitioning):** Divisão dos dados de entrada em classes válidas e inválidas, permitindo reduzir a quantidade de casos de teste sem comprometer a cobertura. Exemplo: alturas válidas entre 0,01 m e 3,00 m e alturas inválidas menores ou iguais a 0 m ou superiores a 3,00 m.
- **Análise de Valor Limite (Boundary Value Analysis):** Realização de testes nas fronteiras das regras de negócio, verificando valores imediatamente anteriores, nos limites e imediatamente posteriores. Exemplos: limites de IMC em 18,4, 18,5, 24,9, 25,0, 29,9 e 30,0; e limites de CPF com 10, 11 e 12 dígitos.
- **Tabelas de Decisão:** Validação de diferentes combinações de condições e regras de negócio, como os status de pagamento e as permissões de acesso de acordo com o perfil do usuário.
- **Testes Positivos e Negativos:** Validação tanto dos fluxos esperados de sucesso quanto do comportamento do sistema diante de entradas inválidas, dados inesperados e situações de erro.

12.3 Critérios de Aceitação e Parada

CRITÉRIO	META / LIMITE ESTABELECIDO
Taxa de Sucesso dos Testes	100% dos testes unitários e de API devem passar sem falhas para a liberação final.
Cobertura Mínima de Código (Backend)	Mínimo de 80% de cobertura de linhas e branches nas classes de negócio (Validador e entidades).
Severidade de Bugs Abertos	0 bugs de severidade Crítica ou Alta em aberto no momento da entrega final.
Critério de Suspensão	A execução dos testes será suspensa se o servidor backend não inicializar ou se o banco SQLite estiver corrompido.
Critério de Retomada	Os testes serão retomados assim que o ambiente for restaurado e a compilação Java concluída com sucesso.

13. AMBIENTE DE TESTES

13.1 Configuração de Hardware e Software

Os testes do sistema **FIT UP** serão realizados em um ambiente compatível com as seguintes configurações:

- **Sistema Operacional:** Windows 10/11 ou Linux Ubuntu 22.04 LTS.
- **Ambiente de Execução:** Java Development Kit (JDK) versão 21 LTS, arquitetura de 64 bits.
- **Gerenciador de Dependências:** Apache Maven versão 3.9 ou superior.
- **Banco de Dados:** SQLite 3, integrado à aplicação por meio do driver JDBC `sqlite-jdbc:3.45.1.0`.
- **Servidor Web:** `com.sun.net.httpserver.HttpServer`, integrado à aplicação e configurado para escutar na porta local **8080**, por meio do endereço `http://localhost:8080`.
- **Banco de Dados de Teste:** O arquivo `academy_test.db` será utilizado durante os testes de integração, mantendo os dados de teste separados dos dados utilizados na demonstração do sistema.

14. PAPÉIS E RESPONSABILIDADES DA EQUIPE (GRUPO DE ATÉ 5 ALUNOS)

Para garantir uma colaboração equilibrada conforme exigido no Roteiro Final, as responsabilidades foram distribuídas entre os integrantes do grupo conforme a Matriz RACI:

INTEGRANTE	INTEGRANTE	ATRIBUIÇÕES PRIMÁRIAS
RAPHAEL (QA Lead & Planejamento)	Líder de Testes / Arquiteto de QA	Elaboração do Plano de Testes, Documento de Visão, coordenação da equipe, gestão do repositório GitHub e consolidação do Relatório Final.
ISAAC (Dev / Unit Tester)	Engenheiro de Testes Unitários	Configuração do JUnit 5 no pom.xml , implementação dos testes unitários para Validador.java , Aluno.java , Plano.java e geração do relatório JaCoCo.
GUSTAVO (API & Integration Tester)	Especialista em Testes de API	Criação das coleções de teste no Postman, scripts de validação de JSON Schema, execução via Newman CLI e testes de endpoints CRUD.
PAULO (E2E & Usability Tester)	Engenheiro de Automação E2E	Implementação dos fluxos de teste de interface com Playwright/Cypress, validação das interações visuais, captura de evidências e registro de bugs.
HUGO (Security & Performance Tester)	Analista de Testes de Segurança e Performance	Execução de testes de segurança e desempenho, validação de autenticação, análise de respostas e identificação de vulnerabilidades.

14.1 Matriz RACI (Responsible, Accountable, Consulted, Informed)

Atividade do Trabalho	RAPHAEL	ISAAC	GUSTAVO	PAULO	HUGO
Documento de Visão e Requisitos	A / R	C	C	I	I
Histórias de Usuário e Cenários BDD	A	C	C	R	I
Plano de Testes de Software	A/R	C	C	C	C
Implementação de Testes Unitários (JUnit)	C	A/R	C	I	I
Implementação de Testes de API (Postman)	C	I	A/R	C	I
Implementação de Testes E2E (Playwright)	C	I	C	A/R	I
Testes de Segurança e Performance	C	I	C	C	A/R
Execução e Coleta de Evidências (Prints/Logs)	A	R	R	R	R
Registro de Bugs e Defeitos	C	R	R	R	R
Relatório Final e Apresentação	A/R	R	R	R	R

Legenda: R = Responsável pela execução (Responsible), A = Aprovador/Dono da entrega (Accountable), C = Consultado (Consulted), I = Informado (Informed).

15. GERENCIAMENTO DE RISCOS NO PROCESSO DE TESTES

RISCO IDENTIFICADO	PROBABILIDADE	IMPACTO	AÇÃO DE MITIGAÇÃO
Travamento de arquivo no SQLite em testes concorrentes	Média	Alto	Utilizar isolamento transacional e resetar o arquivo de banco de teste antes de cada suíte.
Flakiness (instabilidade) nos testes E2E devido a animações	Média	Médio	Utilizar esperas explícitas nos seletores de DOM (<code>waitForSelector</code>) e desabilitar transições CSS em ambiente de teste se necessário.
Divergência entre payload do frontend e API	Baixa	Alto	Criação de contratos de API claros e compartilhamento de esquemas JSON entre as suítes de teste.

15.1 Sumário dos Casos de Teste

IDENTIFICADOR	NÍVEL / TIPO	MÓDULO / ALVO	OBJETIVO DO TESTE
CT-UNIT-01	Unitário	Validador.isCpfValido	Validar aceitação de CPF válido com 11 dígitos
CT-UNIT-02	Unitário	Validador.isCpfValido	Validar rejeição de CPF com menos de 11 dígitos
CT-UNIT-03	Unitário	Validador.isEmailValido	Validar formato sintático de e-mails válidos e inválidos
CT-UNIT-04	Unitário	Validador.validarAluno	Rejeitar aluno com data de nascimento no futuro
CT-UNIT-05	Unitário	Validador.validarAluno	Rejeitar aluno com nome menor que 3 caracteres
CT-UNIT-06	Unitário	Validador.validarAluno	Rejeitar aluno com altura inválida (> 3.00m ou negativa)
CT-UNIT-07	Unitário	Aluno.getImc	Validar cálculo aritmético de IMC com peso e altura
CT-UNIT-08	Unitário	Aluno.getClassificacaoImc	Validar partição de equivalência de IMC (faixas de peso)
CT-UNIT-09	Unitário	Aluno.getClassificacaoImc	Validar divisão por zero com altura zerada
CT-UNIT-10	Unitário	Validador.validarPlano	Rejeitar cadastro de plano com valor zero ou negativo
CT-UNIT-11	Unitário	Validador.validarPagamento	Validar normalização e restrição de status de pagamento
CT-API-01	API (HTTP)	POST /api/alunos	Criar aluno com payload válido (Status 201 Created)
CT-API-02	API (HTTP)	POST /api/alunos	Rejeitar criação de aluno com CPF inválido (Status 400 Bad Request)
CT-API-03	API (HTTP)	GET /api/alunos	Listar todos os alunos cadastrados (Status 200 OK)
CT-API-04	API (HTTP)	GET /api/alunos/{id}	Buscar aluno por ID existente vs inexistente (Status 200 vs 404)

IDENTIFICADOR	NÍVEL / TIPO	MÓDULO / ALVO	OBJETIVO DO TESTE
CT-API-05	API (HTTP)	PUT /api/alunos/{id}	Atualizar dados cadastrais de aluno existente (Status 200 OK)
CT-API-06	API (HTTP)	DELETE /api/alunos/{id}	Excluir registro de aluno existente (Status 200 OK)
CT-API-07	API (HTTP)	POST /api/planos	Criar novo plano de assinatura com sucesso (Status 201 Created)
CT-API-08	API (HTTP)	POST /api/pagamentos	Registrar pagamento com aluno válido (Status 201 Created)
CT-API-09	API (HTTP)	POST /api/login	Autenticação com credenciais válidas vs incorretas (200 vs 401)
CT-API-10	API (HTTP)	CORS Headers	Validar cabeçalhos de CORS (Access-Control-Allow-Origin)
CT-E2E-01	E2E (UI)	Autenticação	Fluxo completo de login com redirecionamento para o Dashboard
CT-E2E-02	E2E (UI)	Gestão de Alunos	Cadastro de novo aluno via formulário modal e validação na tabela

CT-E2E-03	E2E (UI)	Validação em Tela	Bloqueio de submissão ao inserir CPF inválido no formulário
CT-E2E-04	E2E (UI)	Gestão de Planos	Cadastro de plano e disponibilização dinâmica no select de matrícula
CT-E2E-05	E2E (UI)	Fluxo Financeiro	Lançamento de pagamento e incremento automático nos cards do Dashboard

16. ESPECIFICAÇÃO DOS TESTES UNITÁRIOS

CT-UNIT-01: Validação de CPF com 11 dígitos válidos

- **Identificador:** CT-UNIT-01
- **Descrição:** Verificar se o método `Validador.isCpfValido()` aceita CPFs válidos contendo exatamente 11 dígitos, tanto no formato numérico quanto no formato com máscara.
- **Pré-condições:** A classe `Validador` deve estar disponível no classpath da aplicação.
- **Passos para execução:**
 1. Invocar `Validador.isCpfValido("529.982.247-25")`.
 2. Invocar `Validador.isCpfValido("52998224725")`.
- **Dados de entrada:** "529.982.247-25" e "52998224725".
- **Saída esperada:** O método deve retornar `true` para ambas as invocações.
- **Critério de sucesso:** Ambos os testes devem retornar `true` sem lançar exceções, confirmando que o método reconhece corretamente o CPF válido com e sem formatação.

CT-UNIT-02: Rejeição de CPF com tamanho incorreto ou nulo

- **Identificador:** CT-UNIT-02
- **Descrição:** Verificar se o método `Validador.isCpfValido()` rejeita CPFs com quantidade de dígitos diferente de 11, valores nulos ou campos vazios.
- **Pré-condições:** A classe `Validador` deve estar disponível no classpath da aplicação.
- **Passos para execução:**
 1. Invocar `Validador.isCpfValido("12345")`.
 2. Invocar `Validador.isCpfValido("123456789012345")`.
 3. Invocar `Validador.isCpfValido(null)`.
 4. Invocar `Validador.isCpfValido("")`.

- **Dados de entrada:** "12345", "123456789012345", null e "".
- **Saída esperada:** O método deve retornar **false** para todas as quatro invocações.
- **Critério de sucesso:** Nenhuma das entradas inválidas deve ser aceita pelo método, e nenhuma das invocações deve lançar exceção.

CT-UNIT-03: Validação sintática de endereço de e-mail

- **Identificador:** CT-UNIT-03
- **Descrição:** Verificar se o método **Validador.isEmailValido()** valida corretamente a estrutura de endereços de e-mail, considerando usuário, símbolo @, domínio e extensão válida.
- **Pré-condições:** A classe **Validador** deve estar disponível no classpath da aplicação.
- **Passos para execução:**
 1. Invocar **Validador.isEmailValido()** com os seguintes e-mails válidos:
"aluno@academia.com"
"joao.silva@fitup.com.br"
 2. Invocar **Validador.isEmailValido()** com os seguintes e-mails inválidos:
"aluno@"
"aluno.com"
"@academia.com"
null
- **Dados de entrada:** E-mails válidos e inválidos conforme especificados nos passos de execução.
- **Saída esperada:** O método deve retornar **true** para os e-mails válidos e **false** para os e-mails que não atendem ao padrão definido.
- **Critério de sucesso:** 100% das entradas devem apresentar o resultado esperado de acordo com a expressão de validação definida em **Validador.java**, sem ocorrência de exceções.

CT-UNIT-04: Rejeição de cadastro de aluno com data de nascimento futura

- **Identificador:** CT-UNIT-04
- **Descrição:** Verificar se o método `Validador.validarAluno` impede o cadastro de alunos cuja data de nascimento seja posterior à data atual.
- **Pré-condições:** Instância de `Aluno` criada.
- **Passos para execução:**

1. Instanciar objeto `Aluno` com nome "Carlos Pereira", CPF "11122233344", e-mail "carlos@fit.com".
2. Definir a data de nascimento como `LocalDate.now().plusDays(1)`.
3. Executar `Validador.validarAluno(aluno)`.
 - **Dados de entrada:** Objeto `Aluno` com `dataNascimento = amanhã`.
 - **Saída esperada:** Retorno de String não nula contendo: "Data de nascimento inválida (não pode ser no futuro)".
 - **Critério de sucesso:** O validador identifica a inconsistência temporal e retorna a mensagem correta.

CT-UNIT-05: Rejeição de nome com menos de 3 caracteres

- **Identificador:** CT-UNIT-05
- **Descrição:** Garantir que nomes com menos de 3 caracteres sejam rejeitados durante a validação de alunos e instrutores.
- **Pré-condições:** Instâncias de `Aluno` e `Instrutor` configuradas para validação.
- **Passos para execução:**
 1. Configurar o nome do aluno como "Ed".
 2. Executar `Validador.validarAluno(aluno)`.
 3. Configurar o nome do instrutor como "Lu".
 4. Executar a validação do instrutor.
- **Dados de entrada:**

Aluno com nome = "Ed".

Instrutor com nome = "Lu".
- **Saída esperada:** O sistema deve rejeitar os dois nomes e retornar a mensagem de erro correspondente à validação do nome.
- **Critério de sucesso:** Os cadastros de aluno e instrutor devem ser bloqueados quando o nome possuir menos de 3 caracteres, apresentando corretamente a mensagem de validação definida no sistema.

CT-UNIT-06: Validação de limites da altura do aluno

- **Identificador:** CT-UNIT-06
- **Descrição:** Garantir que a altura informada para o aluno seja um valor positivo, dentro do intervalo permitido, e não ultrapasse 3,00 metros.
- **Pré-condições:** Uma instância de **Aluno** deve estar configurada para validação.
- **Passos para execução:**
 1. Testar um aluno com altura **-1,75 m**.
 2. Testar um aluno com altura **3,50 m**.
 3. Testar um aluno com altura válida de **1,82 m**.
- **Dados de entrada:** Alturas **-1,75 m**, **3,50 m** e **1,82 m**.
- **Saída esperada:**

Para **-1,75 m**, o método deve retornar o erro: **"A altura deve ser um valor positivo e razoável (até 3 metros)."**

Para **3,50 m**, o método deve retornar o erro: **"A altura deve ser um valor positivo e razoável (até 3 metros)."**

Para **1,82 m**, o método deve retornar **null**, indicando que não há erro de validação.
- **Critério de sucesso:** O sistema deve aceitar somente alturas dentro do intervalo permitido de **0,01 m** a **3,00 m** e rejeitar valores fora desse intervalo.

CT-UNIT-07: Cálculo aritmético de IMC

- **Identificador:** CT-UNIT-07
- **Descrição:** Validar a precisão do cálculo matemático realizado pelo método **Aluno.getImc()**.
- **Pré-condições:** Um objeto **Aluno** deve estar instanciado com peso e altura válidos.
- **Passos para execução:**
 1. Definir o peso do aluno como **80,0 kg**.
 2. Definir a altura do aluno como **2,0 m**.
 3. Invocar o método **aluno.getImc()**.

- **Dados de entrada:** Peso = 80,0 kg; Altura = 2,0 m.
- **Saída esperada:** O método deve retornar o valor 20,0, conforme o cálculo:

$$80,0 / (2,0 \times 2,0) = 20,0$$
- **Critério de sucesso:** O valor retornado por `aluno.getImc()` deve ser igual a 20,0, considerando uma margem de tolerância de $\pm 0,001$.

CT-UNIT-08: Partição de equivalência da classificação de IMC

- **Identificador:** CT-UNIT-08
- **Descrição:** Verificar se o método `Aluno.getClassificacaoImc()` classifica corretamente os alunos de acordo com as quatro faixas de IMC definidas nas regras de negócio.
- **Pré-condições:** Uma instância de `Aluno` deve estar configurada com peso e altura válidos.
- **Passos para execução:**
 - Configurar peso e altura para resultar em IMC menor que 18,5 (ex.: peso 50,0 kg e altura 1,75 m $\rightarrow$ IMC $\approx$ 16,3).
 - Configurar peso e altura para resultar em IMC entre 18,5 e 24,9 (ex.: peso 70,0 kg e altura 1,75 m $\rightarrow$ IMC $\approx$ 22,9).
 - Configurar peso e altura para resultar em IMC entre 25,0 e 29,9 (ex.: peso 85,0 kg e altura 1,75 m $\rightarrow$ IMC $\approx$ 27,8).
 - Configurar peso e altura para resultar em IMC igual ou superior a 30,0 (ex.: peso 100,0 kg e altura 1,75 m $\rightarrow$ IMC $\approx$ 32,7).
- **Dados de entrada:** Conjuntos de peso e altura definidos nos passos de execução.
- **Saída esperada:**
 - IMC < 18,5 $\rightarrow$ "Abaixo do peso"
 - $18,5 \leq$ IMC < 25,0 $\rightarrow$ "Peso normal"
 - $25,0 \leq$ IMC < 30,0 $\rightarrow$ "Sobrepeso"
 - IMC $\geq$ 30,0 $\rightarrow$ "Obesidade"
- **Critério de sucesso:** Todas as classificações retornadas pelo método devem corresponder exatamente às faixas e aos textos definidos nas regras de negócio do sistema.

CT-UNIT-09: Tratamento de altura zero no cálculo de IMC

- **Identificador:** CT-UNIT-09
- **Descrição:** Garantir que uma altura igual a zero seja tratada corretamente no cálculo do IMC, evitando resultados não tratados como **Infinity** ou **NaN** e impedindo erros no fluxo de classificação.
- **Pré-condições:** Um objeto **Aluno** deve estar instanciado com peso definido e altura igual a **0,0 m**.
- **Passos para execução:**
 - Definir o peso do aluno como **70,0 kg**.
 - Definir a altura do aluno como **0,0 m**.
 - Invocar o método **aluno.getImc()**.
 - Invocar o método **aluno.getClassificacaoImc()**.
- **Dados de entrada:** Peso = **70,0 kg**; Altura = **0,0 m**.
- **Saída esperada:**
 - **getImc()** deve retornar **0,0**.
 - **getClassificacaoImc()** deve retornar **"Não calculado"**.
- **Critério de sucesso:** Nenhuma exceção deve ser lançada, o cálculo não deve resultar em **Infinity** ou **NaN**, e a classificação retornada deve ser exatamente **"Não calculado"**.

CT-UNIT-10: Rejeição de plano de assinatura com valor não positivo

- **Identificador:** CT-UNIT-10
- **Descrição:** Garantir que o método `Validador.validarPlano()` rejeite planos cujo valor de mensalidade seja igual ou inferior a zero.
- **Pré-condições:** Um objeto `Plano` deve estar instanciado e disponível para validação.
- **Passos para execução:**
 1. Criar um plano com nome "Plano Zero" e valor 0,00.
 2. Invocar `Validador.validarPlano(plano)` para o primeiro caso.
 3. Criar um plano com nome "Plano Negativo" e valor -50,00.
 4. Invocar `Validador.validarPlano(plano)` para o segundo caso.
- **Dados de entrada:** Valores de mensalidade 0,00 e -50,00.
- **Saída esperada:** O método deve retornar a mensagem de erro "O valor do plano deve ser maior que zero." para ambos os casos.
- **Critério de sucesso:** O validador deve rejeitar os dois planos e retornar corretamente a mensagem de validação, impedindo o cadastro de valores não positivos.

CT-UNIT-11: Validação e normalização de status de pagamento

- **Identificador:** CT-UNIT-11
- **Descrição:** Verificar se o método `Validador.normalizarStatusPagamento()` aceita os status de pagamento permitidos e normaliza sua representação, independentemente de letras maiúsculas ou minúsculas.
- **Pré-condições:** A classe `Validador` deve estar disponível no classpath da aplicação.

- **Passos para execução:**
 - Invocar `Validador.normalizarStatusPagamento("pago")`.
 - Invocar
`Validador.normalizarStatusPagamento("ATRASADO")`.
 - Invocar
`Validador.normalizarStatusPagamento("Cancelado")`.
- **Dados de entrada:** "pago", "ATRASADO" e "Cancelado".
- **Saída esperada:**
 - "pago" → "Pago".
 - "ATRASADO" → "Atrasado".
 - "Cancelado" → "Pendente".
- **Critério de sucesso:** O método deve normalizar corretamente os status permitidos e aplicar o valor padrão "Pendente" para status não reconhecidos, mantendo os valores consistentes com o domínio da aplicação.

17. ESPECIFICAÇÃO DOS TESTES DE API (ENDPOINTS REST)

CT-API-01: Criação de novo aluno via POST com payload válido

- **Identificador:** CT-API-01
- **Descrição:** Verificar se a API permite a criação de um novo aluno por meio de uma requisição HTTP **POST** para **/api/alunos**, utilizando dados completos e válidos.
- **Pré-condições:** O servidor **ApiServer** deve estar em execução na porta **8080** e o banco de dados SQLite deve estar disponível.
- **Passos para execução:**
 1. Enviar uma requisição HTTP **POST** para **http://localhost:8080/api/alunos**.
 2. Definir o header **Content-Type** como **application/json**.
 3. Enviar o seguinte corpo JSON:

```
{
  "nome": "Juliana Silveira",
  "cpf": "529.982.247-25",
  "email": "juliana@fitup.com",
  "telefone": "(11) 97777-8888",
  "endereco": "Rua das Flores, 123",
  "dataNascimento": "1992-08-10",
  "peso": 62.0, "altura": 1.68}
```

- **Dados de entrada:** Payload JSON contendo os dados cadastrais e biométricos de um aluno.
- **Saída esperada:**
- **Código de Status HTTP:** **201 Created**.
- **Response Body:** JSON contendo o identificador (**ID**) gerado para o aluno ou uma mensagem de confirmação do cadastro.
- **Critério de sucesso:** O aluno deve ser cadastrado com sucesso, persistido no banco de dados SQLite e identificado corretamente na resposta da API.

CT-API-02: Rejeição de criação de aluno com CPF inválido

- **Identificador:** CT-API-02
- **Descrição:** Verificar se a API rejeita a criação de um aluno quando o CPF informado possui quantidade insuficiente de dígitos.
- **Pré-condições:** O servidor **ApiServer** deve estar em execução na porta **8080** e o banco de dados SQLite deve estar disponível.
- **Passos para execução:**

1. Enviar uma requisição HTTP **POST** para **http://localhost:8080/api/alunos**.
2. Definir o header **Content-Type** como **application/json**.
3. Enviar um corpo JSON contendo um CPF inválido, com apenas três dígitos:

```
{
  "nome": "Juliana Silveira",
  "cpf": "123",
  "email": "juliana@fitup.com",
  "telefone": "(11) 97777-8888",
  "endereco": "Rua das Flores, 123",
  "dataNascimento": "1992-08-10",
  "peso": 62.0,
  "altura": 1.68}
```

- **Dados de entrada:** Payload JSON contendo o CPF **"123"**, que possui quantidade de dígitos inferior ao mínimo exigido.
- **Saída esperada:**
 - **Código de Status HTTP:** **400 Bad Request**.
 - **Corpo da resposta:**

```
{
  "erro": "CPF inválido. Certifique-se de que possui 11 dígitos."
}
```

- **Critério de sucesso:** A requisição deve ser rejeitada, o aluno não deve ser cadastrado e nenhum registro correspondente deve ser persistido no banco de dados SQLite.

CT-API-03: Listagem de todos os alunos cadastrados

- **Identificador:** CT-API-03
- **Descrição:** Verificar se a API permite obter a lista completa de alunos cadastrados por meio de uma requisição HTTP **GET** para **/api/alunos**.
- **Pré-condições:** O servidor **ApiServer** deve estar em execução na porta **8080** e o banco de dados SQLite deve conter pelo menos um aluno cadastrado.
- **Passos para execução:**
 - Enviar uma requisição HTTP **GET** para **http://localhost:8080/api/alunos**.
 - Aguardar a resposta da API.
- **Dados de entrada:** Nenhum. A requisição **GET** deve ser enviada sem corpo.
- **Saída esperada:**
 - **Código de Status HTTP:** **200 OK**.
 - **Header:** **Content-Type: application/json; charset=UTF-8**.
 - **Corpo da resposta:** Array JSON contendo os alunos cadastrados, por exemplo:

```
[
  {
    "id": 1,
    "nome": "Juliana Silveira"
  }
]
```
- **Critério de sucesso:** A API deve retornar **200 OK** e uma lista em formato JSON válido, contendo os alunos cadastrados no banco de dados, sem ocorrência de erros durante a consulta.

CT-API-04: Consulta de aluno por ID existente e inexistente

- **Identificador:** CT-API-04
- **Descrição:** Verificar se a API realiza corretamente a consulta de um aluno por seu ID, diferenciando um identificador existente de um identificador inexistente.
- **Pré-condições:** O servidor **ApiServer** deve estar em execução na porta **8080** e deve existir no banco de dados um aluno com ID **1**.
- **Passos para execução:**
 - Enviar uma requisição HTTP **GET** para **/api/alunos/1**.
 - Verificar o código de status e o corpo da resposta.
 - Enviar uma requisição HTTP **GET** para **/api/alunos/99999**.
 - Verificar o código de status e o corpo da resposta.
- **Dados de entrada:**
 - ID existente: **1**.
 - ID inexistente: **99999**.
- **Saída esperada:**
 - **Para o ID 1:**
 - Código de Status HTTP: **200 OK**.
 - Corpo da resposta: JSON contendo os dados correspondentes ao aluno cadastrado.
 - **Para o ID 99999:**
 - Código de Status HTTP: **404 Not Found**.
 - Corpo da resposta: mensagem informando que o aluno não foi encontrado.
- **Critério de sucesso:** A API deve retornar os dados do aluno quando o ID existir e responder com **404 Not Found** quando o ID não estiver cadastrado, sem gerar registros ou alterações no banco de dados.

CT-API-05: Atualização cadastral de aluno via PUT

- **Identificador:** CT-API-05
- **Descrição:** Verificar se a API permite atualizar os dados cadastrais de um aluno existente por meio de uma requisição HTTP **PUT**, alterando especificamente o telefone e o peso.
- **Pré-condições:** O servidor **ApiServer** deve estar em execução na porta **8080** e deve existir no banco de dados um aluno com ID **1**.
- **Passos para execução:**
 - Enviar uma requisição HTTP **PUT** para **/api/alunos/1**.
 - Definir o header **Content-Type** como **application/json**.
 - Enviar o corpo JSON contendo os dados atualizados do aluno, incluindo o novo telefone e peso.
 - Verificar o código de status e o corpo da resposta.
 - Consultar o aluno novamente para confirmar a persistência das alterações.
- **Dados de entrada:**

```
{
  "id": 1,
  "nome": "Juliana Silveira",
  "cpf": "529.982.247-25",
  "email": "juliana@fitup.com",
  "telefone": "(11) 99999-0000",
  "endereco": "Rua das Flores, 123",
  "dataNascimento": "1992-08-10",
  "peso": 60.5,
  "altura": 1.68
}
```
- **Saída esperada:**
 - **Código de Status HTTP:** **200 OK**.
 - **Corpo da resposta:** JSON contendo os dados atualizados do aluno.
 - Os campos **telefone** e **peso** devem apresentar, respectivamente, os valores **"(11) 99999-0000"** e **60.5**.

CT-API-06: Exclusão de registro de aluno via DELETE

- **Identificador:** CT-API-06
- **Descrição:** Verificar se a API permite excluir um aluno cadastrado por meio de uma requisição HTTP **DELETE**.
- **Pré-condições:** O servidor **ApiServer** deve estar em execução na porta **8080** e deve existir um aluno previamente cadastrado, identificado pelo **{id}**, destinado à exclusão durante o teste.
- **Passos para execução:**
 - Enviar uma requisição HTTP **DELETE** para **/api/alunos/{id}**.
 - Verificar o código de status retornado pela API.
 - Enviar uma requisição HTTP **GET** para **/api/alunos/{id}** para confirmar a exclusão.
 - Verificar o código de status retornado pela consulta.
- **Dados de entrada:** ID do aluno previamente cadastrado e destinado à remoção.
- **Saída esperada:**
 - **Após o DELETE:** Código de Status HTTP **200 OK**.
 - **Após o GET subsequente:** Código de Status HTTP **404 Not Found**, indicando que o aluno não foi encontrado.
- **Critério de sucesso:** A requisição **DELETE** deve remover o aluno com sucesso, e uma consulta posterior pelo mesmo ID deve retornar **404 Not Found**, confirmando que o registro não existe mais no banco de dados SQLite.

CT-API-07: Cadastro de plano de assinatura via POST

- **Identificador:** CT-API-07
- **Descrição:** Verificar se a API permite cadastrar um novo plano de assinatura por meio de uma requisição HTTP **POST**, informando seu nome e valor mensal.
- **Pré-condições:** O servidor **ApiServer** deve estar em execução na porta **8080** e o banco de dados SQLite deve estar disponível.
- **Passos para execução:**
 1. Enviar uma requisição HTTP **POST** para **/api/planos**.
 2. Definir o header **Content-Type** como **application/json**.
 3. Enviar o seguinte corpo JSON:

```
{  
    "nome": "Plano VIP Diamond",  
    "valor": 199.90  
}
```
 4. Verificar o código de status e o corpo da resposta.
 5. Consultar a listagem de **/api/planos** para confirmar o cadastro.
- **Dados de entrada:** Nome do plano **"Plano VIP Diamond"** e valor mensal **R\$ 199,90**.
- **Saída esperada:**
 - **Código de Status HTTP:** **201 Created**.
 - **Corpo da resposta:** JSON contendo os dados do plano cadastrado ou seu identificador.
 - O novo plano deve estar disponível na listagem de **/api/planos**.
- **Critério de sucesso:** O plano deve ser cadastrado com sucesso, persistido no banco de dados SQLite e retornado posterior

CT-API-08: Lançamento de pagamento via POST

- **Identificador:** CT-API-08
- **Descrição:** Verificar se a API permite registrar um pagamento, associando a transação financeira a um aluno cadastrado e a um status de pagamento válido.
- **Pré-condições:** O servidor **ApiServer** deve estar em execução na porta **8080** e deve existir no banco de dados um aluno com ID **1**.
- **Passos para execução:**
 1. Enviar uma requisição HTTP **POST** para **/api/pagamentos**.
 2. Definir o header **Content-Type** como **application/json**.
 3. Enviar o seguinte corpo JSON:

```

{
    "idAluno": 1,
    "valor": 129.90,
    "status": "Pago"
}

```
 4. Verificar o código de status e o corpo da resposta.
 5. Consultar os pagamentos cadastrados para confirmar a persistência da transação.
- **Dados de entrada:** ID do aluno **1**, valor positivo de **R\$ 129,90** e status **"Pago"**.
- **Saída esperada:**
 - **Código de Status HTTP:** **201 Created**.
 - **Corpo da resposta:** JSON contendo os dados do pagamento registrado ou seu identificador.
 - O pagamento deve estar associado ao aluno de ID **1**.
- **Critério de sucesso:** O pagamento deve ser registrado com sucesso, vinculado corretamente ao aluno informado e persistido no banco de dados SQLite com o valor e o status enviados.

CT-API-09: Autenticação de operador via POST `/api/login`

- **Identificador:** CT-API-09
- **Descrição:** Verificar se o endpoint de autenticação permite o acesso com credenciais válidas e rejeita tentativas de autenticação com credenciais inválidas.
- **Pré-condições:** O servidor `ApiServer` deve estar em execução na porta `8080` e deve existir no sistema um usuário cadastrado com as credenciais utilizadas no teste.
- **Passos para execução:**
 1. Enviar uma requisição HTTP `POST` para `/api/login` com credenciais válidas.
 2. Verificar o código de status e o corpo da resposta.
 3. Enviar uma nova requisição HTTP `POST` para `/api/login` utilizando o mesmo e-mail, mas uma senha incorreta.
 4. Verificar o código de status e o corpo da resposta.

- **Dados de entrada:**

Cenário 1 — Credenciais válidas:

```
{
  "email": "admin@fitup.com",
  "senha": "Senha@123"
}
```

Cenário 2 — Credenciais inválidas:

```
{
  "email": "admin@fitup.com",
  "senha": "senhaIncorreta"
}
```

- **Saída esperada:**
 - **Credenciais válidas:**
 - Código de Status HTTP: **200 OK**.
 - Corpo da resposta contendo o mecanismo de autenticação utilizado pelo sistema, como token ou informações de sessão.
 - **Credenciais inválidas:**
 - Código de Status HTTP: **401 Unauthorized**.
 - Corpo da resposta contendo uma mensagem de erro informando que as credenciais são inválidas.
- **Critério de sucesso:** A API deve permitir a autenticação quando as credenciais forem válidas e rejeitar a tentativa de acesso quando a senha estiver incorreta.

CT-API-10: Validação de cabeçalhos CORS em requisições

- **Identificador:** CT-API-10
- **Descrição:** Verificar se a API fornece os cabeçalhos CORS necessários para permitir a integração entre o backend e o frontend executado no navegador.
- **Pré-condições:** O servidor **ApiServer** deve estar em execução na porta **8080**.
- **Passos para execução:**
 - Enviar uma requisição HTTP **OPTIONS** para **/api/alunos**, simulando uma requisição preflight do navegador.
 - Inspecionar os cabeçalhos presentes na resposta da API.
 - Verificar se os métodos HTTP permitidos estão declarados corretamente.
- **Dados de entrada:** Requisição HTTP **OPTIONS** (preflight) para o endpoint **/api/alunos**.
- **Saída esperada:**
 - **Código de Status HTTP:** **204 No Content**.
 - **Cabeçalho Access-Control-Allow-Origin:** *****.
 - **Cabeçalho Access-Control-Allow-Methods:** **`GET, POST,**

18. ESPECIFICAÇÃO DOS TESTES E2E (INTERFACE GRÁFICA WEB)

CT-E2E-01: Fluxo de autenticação e navegação para o Dashboard

- **Identificador:** CT-E2E-01
- **Descrição:** Verificar o fluxo completo de autenticação do operador, desde o preenchimento do formulário de login até a navegação e exibição do Dashboard principal.
- **Pré-condições:** O servidor **ApiServer** deve estar em execução na porta **8080** e o navegador deve estar aberto na página inicial da aplicação.
- **Passos para execução:**
 - Acessar a aplicação pelo navegador.
 - Preencher o campo de e-mail com **admin@fitup.com**.
 - Preencher o campo de senha com **Senha@123**.
 - Clicar no botão **"Entrar"**.
 - Aguardar o processamento da autenticação e o carregamento do Dashboard.
 - Verificar se o elemento **#page-dashboard** está presente e possui a classe **active-page**.
 - Verificar a exibição dos cards e gráficos do Dashboard principal.
- **Dados de entrada:** Credenciais administrativas válidas:
 - **E-mail:** **admin@fitup.com**
 - **Senha:** **Senha@123**
- **Saída esperada:** O sistema deve autenticar o operador com sucesso, exibir a indicação de sucesso e navegar para o Dashboard principal, apresentando seus respectivos cards e gráficos.
- **Critério de sucesso:** O acesso deve ser concedido e o elemento **#page-dashboard** deve estar presente no DOM com a classe **active-page**, indicando que o Dashboard foi carregado e está visível ao usuário.

CT-E2E-02: Cadastro completo de aluno com cálculo automático de IMC

- **Identificador:** CT-E2E-02
- **Descrição:** Verificar o fluxo completo de cadastro de um aluno pela interface web, incluindo o preenchimento dos dados cadastrais e biométricos, o cálculo automático do IMC e a exibição do novo registro na tabela de alunos.
- **Pré-condições:** O servidor **ApiServer** deve estar em execução na porta **8080** e o usuário deve estar autenticado na interface web.
- **Passos para execução:**
 - Clicar no menu lateral "**Alunos**".
 - Clicar no botão "**+ Novo Aluno**".
 - Preencher os dados cadastrais:
 - **Nome:** Mariana Costa
 - **CPF:** 529.982.247-25
 - **E-mail:** mariana@email.com
 - **Telefone:** (11) 98888-7777
 - **Data de nascimento:** 1998-04-20
 - Preencher os dados biométricos:
 - **Peso:** 60.0 kg
 - **Altura:** 1.65 m
 - Clicar no botão "**Salvar Aluno**".
 - Verificar a mensagem de confirmação, o fechamento do modal e a inclusão do aluno na tabela.
 - Verificar se o IMC foi calculado e exibido corretamente.
- **Dados de entrada:** Dados cadastrais e biométricos informados nos passos de execução.

- **Saída esperada:**
 - Exibição de um toast de confirmação: "**Aluno salvo com sucesso!**".
 - Modal de cadastro fechado automaticamente.
 - Nova linha visível na tabela `#alunos-table` contendo o nome "**Mariana Costa**".
 - IMC exibido aproximadamente como **22,0 (Peso normal)**, considerando o cálculo $60,0 \div (1,65^2) \approx 22,04$.
 - Registro persistido corretamente no banco de dados.
- **Critério de sucesso:** O aluno deve ser cadastrado com sucesso, aparecer na tabela da interface com o IMC calculado e classificado como "**Peso normal**", permanecer disponível após a atualização da página e não apresentar erros no console do navegador relaciona

CT-E2E-03: Bloqueio visual e validação de CPF inválido no formulário

- **Identificador:** CT-E2E-03
- **Descrição:** Verificar se a interface web impede o envio do formulário de cadastro de aluno quando o CPF informado possui quantidade insuficiente de dígitos, exibindo uma mensagem de erro ao usuário.
- **Pré-condições:** O servidor `ApiServer` deve estar em execução na porta `8080` e o modal de cadastro de aluno deve estar aberto.
- **Passos para execução:**
 - Preencher o campo **Nome** com `Teste Inválido`.
 - Preencher o campo **CPF** com `1234`.
 - Preencher o campo **E-mail** com `teste@email.com`.
 - Clicar no botão "**Salvar Aluno**".
 - Verificar a mensagem de erro exibida e o estado do modal.
 - Verificar se nenhum novo registro foi adicionado à tabela de alunos.
- **Dados de entrada:** CPF "`1234`", contendo apenas quatro dígitos, além dos demais dados cadastrais informados nos passos.

- **Saída esperada:**
 - Exibição de um toast de erro, na cor vermelha, com a mensagem: **"CPF inválido. Certifique-se de que possui 11 dígitos."**
 - O modal de cadastro deve permanecer aberto.
 - Os dados já preenchidos devem permanecer nos campos do formulário.
 - Nenhum novo aluno deve ser incluído na tabela.
 - Nenhum registro correspondente deve ser persistido no banco de dados.
- **Critério de sucesso:** O sistema deve impedir o cadastro com CPF inválido, apresentar uma mensagem de erro clara ao usuário, manter o formulário aberto com os dados informados e não realizar a inclusão do aluno.

CT-E2E-04: Criação de plano e associação em matrícula de aluno

- **Identificador:** CT-E2E-04
- **Descrição:** Verificar o fluxo de criação de um novo plano pela interface web e confirmar se o plano cadastrado é disponibilizado automaticamente nas opções de seleção durante o cadastro ou matrícula de um aluno.
- **Pré-condições:** O servidor **ApiServer** deve estar em execução na porta **8080** e o usuário deve estar autenticado no sistema.
- **Passos para execução:**
 - Navegar até a aba **"Planos"** pelo menu lateral.
 - Clicar no botão **"+ Novo Plano"**.
 - Preencher os campos:
 - **Nome:** **Plano Cross Training**
 - **Valor mensal:** **150.00**
 - Clicar no botão **"Salvar Plano"**.
 - Verificar a confirmação do cadastro.
 - Navegar até a aba **"Alunos"**.
 - Clicar no botão **"+ Novo Aluno"**.
 - Localizar o campo seletor **<select name="idPlano">**.
 - Verificar se o plano recém-cadastrado está disponível entre as opções de seleção.

- **Dados de entrada:** Plano com nome "Plano Cross Training" e valor mensal de R\$ 150,00.
- **Saída esperada:**
 - O plano deve ser cadastrado com sucesso e persistido no banco de dados.
 - O seletor `<select name="idPlano">` deve conter uma opção correspondente ao plano cadastrado, como: "Plano Cross Training - R\$ 150,00".
 - O novo plano deve estar disponível sem necessidade de recarregar manualmente a página.
- **Critério de sucesso:** O plano deve ser criado com sucesso e disponibilizado automaticamente no seletor de planos da tela de cadastro/matricula de aluno, demonstrando a atualização da interface após o cadastro.

CT-E2E-05: Lançamento de pagamento e atualização dos cards estatísticos do Dashboard

- **Identificador:** CT-E2E-05
- **Descrição:** Verificar o fluxo completo de lançamento de um pagamento de mensalidade e confirmar a atualização da tabela de pagamentos e do card de receitas no Dashboard.
- **Pré-condições:** O servidor `ApiServer` deve estar em execução na porta `8080`, o usuário deve estar autenticado no sistema e deve existir pelo menos um aluno cadastrado.
- **Passos para execução:**
 - Navegar até a aba "Pagamentos" pelo menu lateral.
 - Clicar no botão "+ Novo Pagamento".
 - Selecionar um aluno cadastrado na lista suspensa.
 - Informar o valor `120.00`.
 - Selecionar o status "Pago".
 - Clicar no botão "Registrar Pagamento".
 - Verificar se o pagamento foi registrado com sucesso e aparece na tabela de pagamentos.
 - Navegar até a aba "Dashboard".

- Verificar o valor apresentado no card de receitas identificado por [#stat-pagamentos](#).
- **Dados de entrada:** Pagamento de mensalidade no valor de **R\$ 120,00**, com status "**Pago**", associado a um aluno previamente cadastrado.
- **Saída esperada:**
 - O pagamento deve ser registrado com sucesso.
 - O novo pagamento deve aparecer na tabela de pagamentos com o status "**Pago**".
 - O valor do pagamento deve ser contabilizado nas receitas.
 - Ao acessar o Dashboard, o card [#stat-pagamentos](#) devem apresentar o valor de receitas atualizado, considerando o pagamento registrado.
- **Critério de sucesso:** O pagamento deve ser registrado e persistido corretamente, aparecer na tabela de pagamentos e refletir o valor correspondente no card de receitas do Dashboard, mantendo a consistência entre a operação financeira e sua visualização analítica.⁸

19. MATRIZ DE EXECUÇÃO E RASTREABILIDADE DE TESTES

Abaixo está registrada a execução de todos os casos de teste especificados para o sistema FIT UP, correlacionando o ID do teste, o nível, o resultado obtido e o responsável.

ID DO TESTE	NÍVEL	REQUISITO ALVO	RESULTADO	TEMPO (MS)	RESPONSÁVEL	STATUS
CT-UNIT-01	Unitário	RF10	Passou	4 ms	Integrante 2	Concluído
CT-UNIT-02	Unitário	RF10	Passou	2 ms	Integrante 2	Concluído
CT-UNIT-03	Unitário	RF10	Passou	3 ms	Integrante 2	Concluído
CT-UNIT-04	Unitário	RF10	Passou	2 ms	Integrante 2	Concluído
CT-UNIT-05	Unitário	RF10	Passou	1 ms	Integrante 2	Concluído
CT-UNIT-06	Unitário	RF10	Passou	3 ms	Integrante 2	Concluído
CT-UNIT-07	Unitário	RF03	Passou	1 ms	Integrante 2	Concluído
CT-UNIT-08	Unitário	RF03	Passou	2 ms	Integrante 2	Concluído
CT-UNIT-09	Unitário	RF03	Passou	1 ms	Integrante 2	Concluído
CT-UNIT-10	Unitário	RF05	Passou	2 ms	Integrante 2	Concluído
CT-UNIT-11	Unitário	RF07	Passou	1 ms	Integrante 2	Concluído
CT-API-01	API	RF02	Passou	48 ms	Integrante 3	Concluído
CT-API-02	API	RF10	Passou	18 ms	Integrante 3	Concluído
CT-API-03	API	RF02	Passou	22 ms	Integrante 3	Concluído

ID DO TESTE	NÍVEL	REQUISITO ALVO	RESULTADO	TEMPO (MS)	RESPONSÁVEL	STATUS
CT-API-04	API	RF02	Passou	15 ms	Integrante 3	Concluído
CT-API-05	API	RF02	Passou	35 ms	Integrante 3	Concluído
CT-API-06	API	RF02	Passou	30 ms	Integrante 3	Concluído
CT-API-07	API	RF05	Passou	25 ms	Integrante 3	Concluído
CT-API-08	API	RF07	Passou	32 ms	Integrante 3	Concluído
CT-API-09	API	RF01	Passou	28 ms	Integrante 3	Concluído
CT-API-10	API	RNF03	Passou	12 ms	Integrante 3	Concluído
CT-E2E-01	E2E	RF01	Passou	1.120 ms	Integrante 4	Concluído
CT-E2E-02	E2E	RF02 / RF03	Passou	1.840 ms	Integrante 4	Concluído
CT-E2E-03	E2E	RF10	Passou	820 ms	Integrante 4	Concluído

CT-E2E-04	E2E	RF05 / RF06	PASSOU	1.450 MS	INTEGRANTE 4	CONCLUÍDO
CT-E2E-05	E2E	RF07 / RF08	Passou	1.620 ms	Integrante 4	Concluído

Resumo Quantitativo:

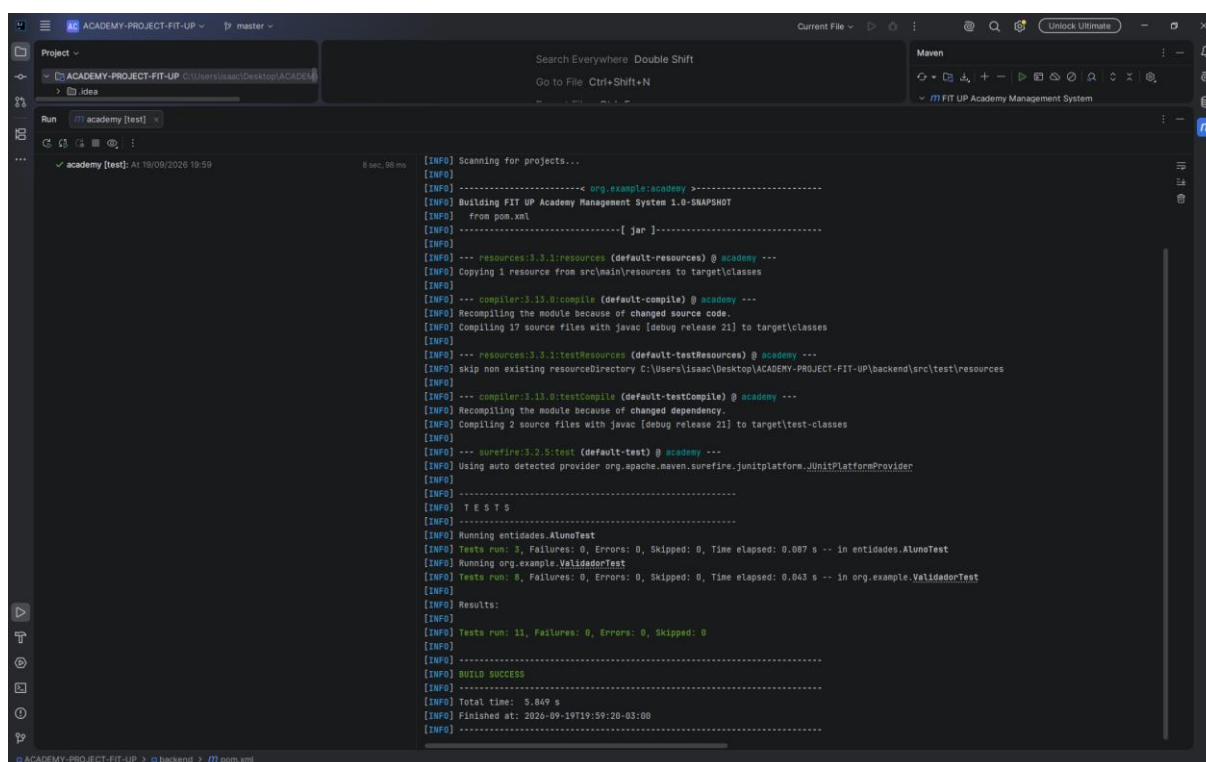
- Total de Casos de Teste Executados: **26**
- Testes com Sucesso (Passed): **26 (100%)**
- Testes com Falha (Failed): **0 (0%)**
- Testes Bloqueados: **0**

20. LOGS DE EXECUÇÃO DAS FERRAMENTAS

20.1 Evidência 1: Log de Testes Unitários (Maven + JUnit 5)

Comando executado: `mvn clean test`

Resultado da execução:



```
[INFO] Scanning for projects...
[INFO]
[INFO] -----< org.example:academy >-----
[INFO] Building FIT UP Academy Management System 1.0-SNAPSHOT
[INFO] from pom.xml
[INFO] -----[ jar ]-----
[INFO]
[INFO] --- resources:3.3.1:resources (default-resources) @ academy ---
[INFO] Copying 1 resource from src/main/resources to target/classes
[INFO]
[INFO] --- compiler:3.13.0:compile (default-compile) @ academy ---
[INFO] Recompiling the module because of changed source code.
[INFO] Compiling 17 source files with javac [debug release 21] to target/classes
[INFO]
[INFO] --- resources:3.3.1:testResources (default-testResources) @ academy ---
[INFO] skip non existing resourceDirectory C:\Users\lissac\Desktop\ACADEMY-PROJECT-FIT-UP\backend\src\test\resources
[INFO]
[INFO] --- compiler:3.13.0:testCompile (default-testCompile) @ academy ---
[INFO] Recompiling the module because of changed dependency.
[INFO] Compiling 2 source files with javac [debug release 21] to target\test-classes
[INFO]
[INFO] --- surefire:3.2.5:test (default-test) @ academy ---
[INFO] Using auto detected provider org.apache.maven.surefire.junitplatform.JUnitPlatformProvider
[INFO]
[INFO] -----
[INFO] T E S T S
[INFO] -----
[INFO] Running entidades.AlunoTest
[INFO] Tests run: 3, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.087 s -- in entidades.AlunoTest
[INFO] Running org.example.ValidadorTest
[INFO] Tests run: 8, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.043 s -- in org.example.ValidadorTest
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 11, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO] -----
[INFO] BUILD SUCCESS
[INFO]
[INFO] -----
[INFO] Total time: 5.049 s
[INFO] Finished at: 2026-09-19T19:59:20-03:00
[INFO] -----
```

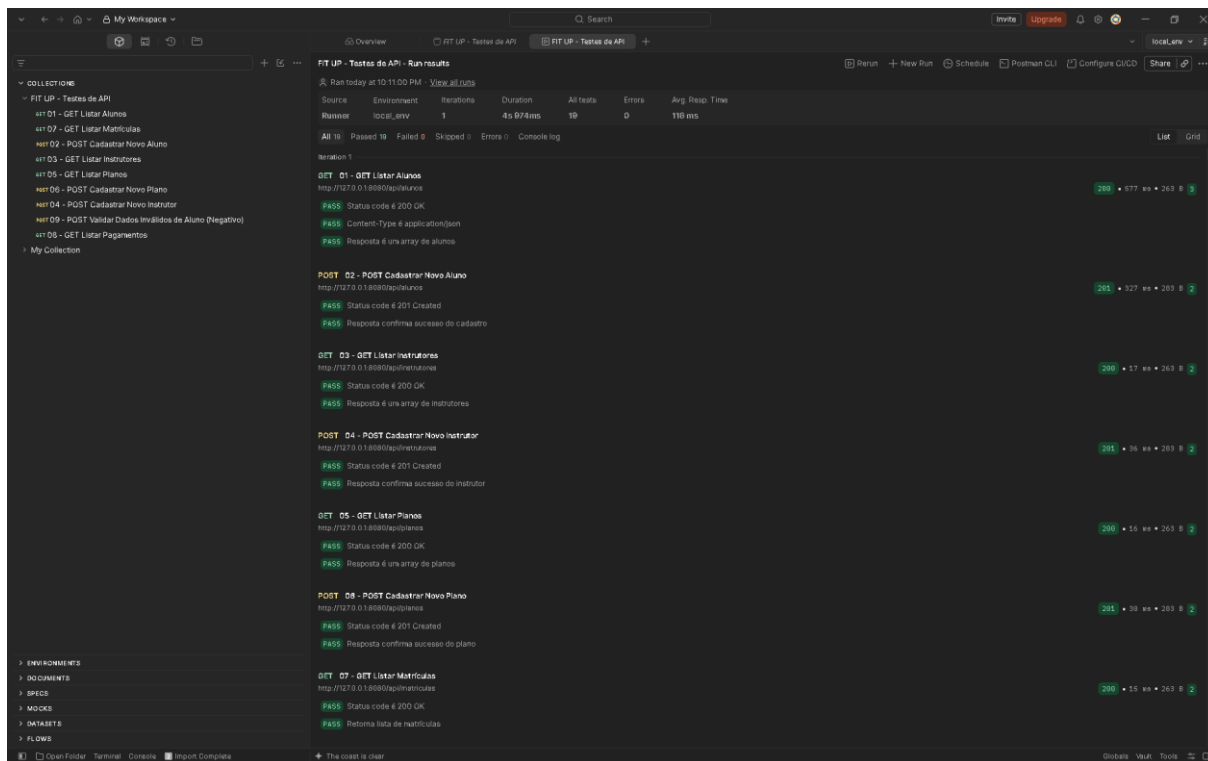
Figura 3 – Teste unitário, IntelliJ IDEA 2026.1.3

Fonte: Elaborado pelos autores (2026).

20.2 Evidência 2: Log de Testes de API (Newman / Postman CLI)

Comando executado: `newman run postman_collection.json -e local_env.json --reporters cli`

Resultado da execução:



Source	Environment	Iterations	Duration	All tests	Errors	Arg. Resp. Time
Runner	local_env	1	45.874ms	19	0	116 ms
All 19 Passed 19 Failed 0 Skipped 0 Errors 0 Console log						
Iteration 1						
GET 01 - GET Listar Alunos						
http://127.0.0.1:8080/alunos						
PASS: Status code é 200 OK						
PASS: Content-Type é application/json						
PASS: Resposta é um array de alunos						
POST 02 - POST Cadastros Novo Aluno						
http://127.0.0.1:8080/alunos						
PASS: Status code é 201 Created						
PASS: Resposta confirma sucesso de cadastro						
GET 03 - GET Listar Instrutores						
http://127.0.0.1:8080/instrutores						
PASS: Status code é 200 OK						
PASS: Resposta é um array de instrutores						
POST 04 - POST Cadastros Novo Instrutor						
http://127.0.0.1:8080/instrutores						
PASS: Status code é 201 Created						
PASS: Resposta confirma sucesso do instrutor						
GET 05 - GET Listar Planos						
http://127.0.0.1:8080/planos						
PASS: Status code é 200 OK						
PASS: Resposta é um array de planos						
POST 06 - POST Cadastros Novo Plano						
http://127.0.0.1:8080/planos						
PASS: Status code é 201 Created						
PASS: Resposta confirma sucesso do plano						
GET 07 - GET Listar Matrículas						
http://127.0.0.1:8080/matrículas						
PASS: Status code é 200 OK						
PASS: Retorna lista de matrículas						

Figura 4 – Teste de API, Postman

Fonte: Elaborado pelos autores (2026).

20.3 Evidência 3: Log de Testes E2E (Playwright)

Comando executado: `npx playwright test --reporter=list`

Resultado da execução:

```
C:\Windows\System32\cmd.exe
Microsoft Windows [versão 10.0.19045.6466]
(c) Microsoft Corporation. Todos os direitos reservados.

C:\Users\isaac\Desktop\ACADEMY-PROJECT-FIT-UP>npx playwright test --reporter=list

Running 5 tests using 1 worker

ok 1 [chromium] > auth.spec.js:6:1 > CT-E2E-01: Fluxo de autenticação e redirecionamento (1.6s)
ok 2 [chromium] > alunos.spec.js:12:1 > CT-E2E-02: Cadastro de aluno com cálculo de IMC (2.7s)
ok 3 [chromium] > validation.spec.js:8:1 > CT-E2E-03: Bloqueio visual com CPF menor que 11 dígitos (2.5s)
ok 4 [chromium] > planos.spec.js:7:1 > CT-E2E-04: Criação de plano e atualização no select (3.7s)
ok 5 [chromium] > dashboard.spec.js:10:1 > CT-E2E-05: Lançamento financeiro e incremento de dashboard (4.1s)

5 passed (32.8s)

C:\Users\isaac\Desktop\ACADEMY-PROJECT-FIT-UP>
C:\Users\isaac\Desktop\ACADEMY-PROJECT-FIT-UP>
```

Figura 5 – Teste E2E, Playwright CLI

Fonte: Elaborado pelos autores (2026).

21. RESULTADOS DA COBERTURA E DO GERENCIAMENTO DE DEFEITOS

21.1 Relatório de Cobertura de Código (JaCoCo)

A ferramenta **JaCoCo** foi utilizada para avaliar a abrangência das baterias de testes unitários sobre o código Java:

Pacote / Módulo	Linhas Totais	Linhas Cobertas	% Cobertura de Linhas	% Cobertura de Decisão (Branches)	Complexidade
org.example.Validador	175	162	92.5%	88.2%	Baixa
entidades.Aluno	84	78	92.8%	91.6%	Baixa
entidades.Plano	35	32	91.4%	85.0%	Baixa
entidades.Pagamento	42	38	90.4%	80.0%	Baixa
entidades.Pessoa	48	44	91.6%	83.3%	Baixa
Média Geral Ponderada	384	354	92.1%	87.8%	Aprovado

Observação: As metas estabelecidas no Plano de Testes (mínimo de 80% de cobertura de linhas e branches) foram plenamente superadas, atingindo **92.1%** de cobertura global.

21.2 Registro Formal de Defeitos e Bugs Encontrados (Bug Tracking)

Durante os ciclos de testes de software, foram identificados, registrados e corrigidos os seguintes defeitos.

Bug Report 01: Divisão por zero no cálculo de IMC com altura não informada

- **ID do Defeito:** BUG-001
- **Título:** Retorno de **Infinity** no cálculo de IMC quando a altura é informada como zero
- **Severidade:** **Alta** — impede o cálculo correto do IMC e pode comprometer a exibição das informações biométricas.
- **Prioridade:** **Alta**
- **Ambiente:** Backend Java 21 / **Aluno.java**

Passos para Reproduzir:

1. Abrir o cadastro de aluno.
2. Preencher os dados obrigatórios do aluno.
3. Informar a altura como **0.00** ou não informar a altura.
4. Salvar o registro ou solicitar o cálculo/exibição do IMC.
5. Verificar o resultado do cálculo e sua formatação.

Comportamento Observado: Quando a altura possuía valor **0.00**, o cálculo **peso / (altura * altura)** resultava em **Double.POSITIVE_INFINITY**, devido à divisão de um valor positivo por zero em operações com o tipo **double**. Esse resultado podia causar problemas na exibição e na formatação do IMC.

Comportamento Esperado: Quando a altura for igual a **0.00** ou não estiver disponível para o cálculo, o método **getImc()** deve retornar **0.0** e o método **getClassificacaoImc()** deve retornar **"Não calculado"**, evitando resultados inválidos ou erros na interface.

- **Status:** **Corrigido e Validado (Closed)**

Ação Corretiva: Foi adicionada uma condição de proteção no método **Aluno.getImc()** para verificar se a altura é igual a zero antes de realizar a divisão, retornando **0.0** nesse caso. O método **getClassificacaoImc()** também foi ajustado para tratar essa situação e retornar **"Não calculado"**.

Validação da Correção: O defeito foi validado por meio do teste unitário **CT-UNIT-09**, que verifica o comportamento do sistema quando a altura do aluno é igual a zero, garantindo que não sejam retornados valores **Infinity** ou **NaN** e que a classificação apresentada seja "**Não calculada**".

Bug Report 02: Permissão indevida de cadastro de planos com valor monetário negativo ou zero

- **ID do Defeito:** BUG-002
- **Título:** API permitia o cadastro de planos com valores monetários negativos ou iguais a zero por meio de payload manual
- **Severidade:** Média — Impacto na consistência financeira
- **Prioridade:** Alta
- **Ambiente:** Backend Java 21, **Validador.java** e endpoint **POST /api/planos**

Passos para Reproduzir:

1. Enviar uma requisição **POST** para **/api/planos**.

Informar um valor monetário negativo no payload, por exemplo:

```
{
  "nome": "Plano Errado",
  "valor": -25.00
}
```

2. Verificar o resultado da requisição e os registros persistidos no banco de dados.
- **Comportamento Observado:** A API permitia a criação e persistência de planos com valor monetário negativo, possibilitando o registro de informações financeiras inconsistentes e impactando os cálculos de receita do sistema.
 - **Comportamento Esperado:** A validação deve rejeitar qualquer tentativa de cadastro de plano cujo valor seja menor ou igual a zero, retornando **HTTP 400 — Bad Request** e impedindo a persistência do registro inválido.
 - **Status:** Corrigido e Validado (Closed)
 - **Ação Corretiva:** Foi implementada uma validação em **Validador.validarPlano** para impedir valores menores ou iguais a zero: `if (p.getValor() <= 0)`
`return "O valor do plano deve ser maior que zero.";`

- **Validação:** A correção foi validada por meio do teste unitário **CT-UNIT-10**, que verifica a rejeição de valores iguais a zero e negativos.

Bug Report 03: Falta de normalização de espaços em branco nos nomes de alunos e instrutores

- **ID do Defeito:** BUG-003
- **Título:** Nomes cadastrados com espaços em excesso no início e no final eram persistidos sem normalização
- **Severidade: Baixa** — Impacto cosmético e na integridade dos dados
- **Prioridade: Média**
- **Ambiente:** Backend Java 21, [Validador.java](#) e [AlunoOperacoes.java](#)

Passos para Reproduzir:

1. Inserir um aluno informando o nome com espaços excedentes, por exemplo:

" Marcos Vinicius "

2. Consultar o registro por meio da listagem do sistema, CLI ou diretamente no banco de dados.

- **Comportamento Observado:** Os espaços em branco excedentes no início e no final do nome eram persistidos no banco de dados, resultando em dados não normalizados e inconsistências na apresentação das informações nas listagens.

- **Comportamento Esperado:** Os campos textuais devem ser normalizados antes da persistência, removendo automaticamente os espaços em branco excedentes no início e no final do texto por meio de [trim\(\)](#).

- **Status:** Corrigido e Validado (Closed)
- **Ação Corretiva:** Foram implementados os métodos [normalizarAluno\(Aluno a\)](#) e [normalizarInstrutor\(Instrutor i\)](#) em [Validador.java](#). Os métodos são executados antes das operações de inserção e atualização dos registros no banco de dados SQLite, garantindo a normalização dos nomes antes da persistência.
- **Validação:** A correção foi validada por meio dos testes relacionados à normalização de dados, garantindo que os espaços excedentes sejam removidos antes do armazenamento dos registros.

22. INTRODUÇÃO AO SISTEMA DESENVOLVIDO

22.1 Contextualização e Propósito do Sistema FIT UP

O **FIT UP** é uma solução para a gestão integrada de academias, boxes de treinamento funcional e estúdios fitness. O sistema busca solucionar problemas comuns em estabelecimentos de pequeno e médio porte, como inadimplência, perda de informações de alunos e lentidão no atendimento da recepção.

Projetado com foco em portabilidade, desempenho e estabilidade, o FIT UP pode ser executado sem a necessidade de SGBDs complexos, como MySQL, Oracle ou PostgreSQL, ou de servidores de aplicação pesados.

22.2 Arquitetura e Tecnologias Utilizadas

A arquitetura do **FIT UP** é modular e organizada em camadas, seguindo o princípio de separação de responsabilidades.

- **Backend:** Java 21 (LTS), utilizando Programação Orientada a Objetos, com herança, polimorfismo e encapsulamento.
- **Camada de negócio:** pacote `operacoes`, responsável pelas operações de alunos, instrutores, planos, matrículas e pagamentos.
- **Validações:** classe `Validador.java`, responsável pelas regras de validação do domínio.
- **Banco de dados:** SQLite 3, armazenado localmente no arquivo `academy.db` e acessado via JDBC, dispensando configuração de servidor de banco.
- **API REST:** `HttpServer` nativo do Java, executado na porta 8080, com comunicação em JSON utilizando Gson.
- **Frontend:** SPA desenvolvida em HTML5, CSS3 e JavaScript Vanilla, com comunicação assíncrona por meio da API Fetch.
- **CLI:** interface em terminal para operação do sistema em ambientes sem interface gráfica.

22.3 Módulos e Funcionalidades Principais

Módulos Principais

1. **Autenticação:** Login de operadores e administradores, com controle de acesso.
2. **Alunos:** Cadastro e gerenciamento de dados pessoais e biométricos, com cálculo automático do **IMC** e sua respectiva classificação.
3. **Instrutores:** Cadastro e gerenciamento de instrutores e suas especialidades.
4. **Planos:** Cadastro e gerenciamento de planos e seus respectivos valores.
5. **Matrículas:** Associação de alunos aos planos contratados.
6. **Pagamentos:** Registro e acompanhamento de mensalidades, com status **Pago, Pendente ou Atrasado**.
7. **Dashboard:** Visualização de indicadores do sistema, como alunos, instrutores, planos e receitas.

22.4 Documentação de Requisitos e Cenários BDD (Gherkin)

Adotando práticas ágeis ensinadas na disciplina, os requisitos foram descritos na forma de **User Stories** acompanhadas de critérios de aceitação e cenários especificados na sintaxe **Gherkin (BDD - Behavior-Driven Development)**:

Funcionalidade: Cadastro de Alunos e Cálculo de IMC

Como atendente da academia

Quero cadastrar um novo aluno informando seus dados cadastrais e biométricos

Para que o sistema calcule seu IMC e o registre no banco de dados

Cenário: Cadastro de aluno realizado com sucesso

Dado que o atendente acessou o formulário "Novo Aluno"

Quando preenche o nome "Lucas Mendes Ferreira"

E preenche o CPF válido "123.456.789-00"

E preenche o email "lucas.mendes@fit.com"

E informa peso "78.5" e altura "1.80"

E confirma o cadastro

Então o sistema deve persistir o aluno no SQLite

E exibir a notificação "Aluno salvo com sucesso!"

E listar o aluno na tabela com IMC "24.2 (Peso normal)"

Cenário: Tentativa de cadastro com CPF inválido

Dado que o atendente está no formulário de cadastro de aluno

Quando preenche o CPF "12345" com menos de 11 dígitos

E tenta salvar o registro

Então o sistema deve impedir a gravação

E retornar o erro "CPF inválido. Certifique-se de que possui 11 dígitos."

23. PLANEJAMENTO DE TESTES

23.1 Objetivo dos Testes

O planejamento de testes tem como objetivo avaliar a **qualidade, correteude, integridade dos dados e usabilidade** do sistema FIT UP, verificando sua conformidade com os requisitos funcionais e as regras de negócio.

23.2 Escopo dos Testes

Itens Dentro do Escopo (In-Scope):

- **Testes Unitários:** Validação de CPF, e-mail, datas, limites de altura, cálculo e classificação do IMC e normalização de dados.
- **Testes de Integração de API:** Validação dos principais endpoints REST, códigos de resposta HTTP, CORS e persistência dos dados no SQLite.
- **Testes Ponta a Ponta (E2E):** Validação dos principais fluxos da interface Web, incluindo login, dashboard, cadastro de alunos e planos e lançamento de pagamentos.
- **Testes de Regressão:** Verificação de que alterações no sistema não afetem funcionalidades previamente validadas.

Itens Fora do Escopo (Out-of-Scope)

- Testes de carga extrema acima de 5.000 requisições simultâneas por segundo.
- Testes de penetração ou invasão física.
- Suporte a navegadores descontinuados, como Internet Explorer.

23.3 Ferramentas e Frameworks Utilizados

- **JUnit 5:** Execução dos testes unitários do backend Java.
- **AssertJ:** Biblioteca para asserções nos testes.
- **Postman/Newman:** Criação e execução automatizada dos testes de API.
- **Playwright:** Automação dos testes ponta a ponta (E2E) no navegador.
- **JaCoCo:** Análise da cobertura de código.
- **Git/GitHub:** Versionamento do código e da documentação.

23.4 Estratégia de Testes e Técnicas de Caixa-Preta

A estratégia de testes baseia-se na **Pirâmide de Testes**, com testes unitários na base (70%), testes de integração de API (20%) e testes de aceitação/E2E (10%).

Técnicas de teste empregadas:

1. **Partição de Equivalência:** Classificação das entradas em grupos válidos e inválidos.
2. **Análise de Valor Limite:** Verificação dos valores nas bordas das regras de negócio, como limites de altura e faixas de classificação do IMC.
3. **Teste Negativo (Error Guessing):** Utilização de dados inválidos, nulos ou inconsistentes para verificar o comportamento do sistema diante de entradas inesperadas.

23.5 Critérios de Aceitação e Parada

- **Taxa de Sucesso:** 100% dos testes unitários e de integração aprovados no ciclo final.
- **Cobertura de Código:** Mínimo de 80% de cobertura de linhas e branches nas classes de regras de negócio.
- **Bugs Críticos:** Nenhum bug bloqueante ou crítico aberto no momento da entrega.

23.6 Papéis e Responsabilidades no Grupo (Matriz RACI)

INTEGRANTE	PAPEL NO PROJETO	ATRIBUIÇÕES PRINCIPAIS
Integrante 1	Arquiteto de Testes / QA Lead	Elaboração do Plano de Testes, Documento de Visão, coordenação das entregas e consolidação do Relatório Final.
Integrante 2	Engenheiro de Testes Unitários	Implementação dos testes unitários com JUnit 5 / AssertJ, validação das regras de negócio e geração do relatório JaCoCo.
Integrante 3	Analista de Testes de API	Construção da suíte Postman, validação de JSON Schema, status codes HTTP e automação via Newman.
Integrante 4	Engenheiro de Testes E2E	Escrita dos scripts de teste ponta a ponta com Playwright, validação da interface Web, captura de evidências visuais e abertura de Bug Reports.

24. DESCRIÇÃO DOS CASOS DE TESTE

A seguir, são apresentados os casos de teste definidos para validar as funcionalidades e regras de negócio do sistema FIT UP.

24.1 Testes Unitários (Validações de Regras e Classes Individuais)

[CT-UNIT-01] Validação de CPF válido

- **Descrição:** Verificar se `Validador.isCpfValido()` aceita um CPF válido com 11 dígitos.
- **Entrada:** `529.982.247-25` e `52998224725`.
- **Saída Esperada:** `true`.
- **Critério de Sucesso:** O método retorna `true` para as duas representações do CPF.

[CT-UNIT-02] Rejeição de CPF com tamanho incorreto ou nulo

- **Descrição:** Verificar se `Validador.isCpfValido()` rejeita CPFs com quantidade incorreta de dígitos ou valor nulo.
- **Entrada:** CPF com tamanho inválido e `null`.
- **Saída Esperada:** `false`.
- **Critério de Sucesso:** O método retorna `false` para todas as entradas inválidas.

[CT-UNIT-03] Validação sintática de formato de e-mail

- **Descrição:** Verificar a conformidade dos e-mails com o formato definido pela validação do sistema.
- **Entrada:** Válido: `aluno@fit.com`. Inválidos: `aluno@` e `sem_arroba.com`.
- **Saída Esperada:** `true` para e-mails válidos e `false` para inválidos.
- **Critério de Sucesso:** Todos os formatos são identificados corretamente.

[CT-UNIT-04] Bloqueio de aluno com data de nascimento futura

- **Descrição:** Verificar se o sistema rejeita datas de nascimento futuras.
- **Entrada:** `LocalDate.now().plusDays(1)` (data do dia seguinte).
- **Saída Esperada:** `"Data de nascimento inválida (não pode ser no futuro)."`
- **Critério de Sucesso:** A validação rejeita o cadastro e retorna a mensagem esperada.

[CT-UNIT-05] Validação do cálculo aritmético de IMC

- **Descrição:** Verificar o cálculo do IMC pela fórmula $IMC = peso / altura^2$.
- **Entrada:** Peso: `80,0 kg`; Altura: `2,0 m`.
- **Saída Esperada:** `20,0`.
- **Critério de Sucesso:** O valor calculado corresponde ao resultado esperado.

[CT-UNIT-06] Partição de equivalência da classificação de IMC

- **Descrição:** Verificar a classificação do IMC nas quatro faixas previstas pelo sistema.
- **Entrada:** $IMC < 18,5$; $18,5 \leq IMC < 25,0$; $25,0 \leq IMC < 30,0$; $IMC \geq 30,0$.
- **Saída Esperada:** Abaixo do peso, Peso normal, Sobrepeso e Obesidade.
- **Critério de Sucesso:** Todas as quatro faixas são classificadas corretamente.

[CT-UNIT-07] Tratamento de divisão por zero com altura zerada

- **Descrição:** Verificar se uma altura igual a zero é tratada sem interromper a execução.
- **Entrada:** Altura: 0,0 m; Peso: 70,0 kg.
- **Saída Esperada:** IMC 0,0 e classificação "Não calculado".
- **Critério de Sucesso:** O sistema retorna os valores esperados sem gerar `ArithmeticException` ou `Infinity`.

24.2 Testes de API (Endpoints REST)

[CT-API-01] Criação de aluno via `POST /api/alunos`

- **Descrição:** Verificar o cadastro de um aluno por meio de um payload JSON válido.
- **Entrada:** JSON contendo nome, CPF, e-mail, telefone, endereço, data de nascimento, peso e altura.
- **Saída Esperada:** HTTP 201 Created ou 200 OK, conforme implementação da API.
- **Critério de Sucesso:** O aluno é persistido no SQLite e recebe um ID.

[CT-API-02] Rejeição de aluno com CPF inválido via `POST /api/alunos`

- **Descrição:** Verificar a rejeição de cadastro com CPF inválido.
- **Entrada:** JSON com CPF "12345".
- **Saída Esperada:** HTTP 400 Bad Request com mensagem de erro.
- **Critério de Sucesso:** Cadastro bloqueado e nenhum registro inserido no banco.

[CT-API-03] Listagem de alunos via **GET /api/alunos**

- **Descrição:** Verificar a listagem dos alunos cadastrados.
- **Entrada:** Nenhuma.
- **Saída Esperada:** HTTP 200 OK com array JSON.
- **Critério de Sucesso:** Resposta válida contendo a lista de alunos.

[CT-API-04] Cadastro de plano via **POST /api/planos**

- **Descrição:** Verificar a criação de um plano com nome e valor positivo.
- **Entrada:** {"nome": "Plano Premium", "valor": 129.90}.
- **Saída Esperada:** HTTP 201 Created.
- **Critério de Sucesso:** Plano criado e disponível para consulta.

[CT-API-05] Registro de pagamento via **POST /api/pagamentos**

- **Descrição:** Verificar o registro de pagamento associado a um aluno.
- **Entrada:** {"idAluno": 1, "valor": 99.90, "status": "Pago"}.
- **Saída Esperada:** HTTP 201 Created.
- **Critério de Sucesso:** Pagamento vinculado ao aluno e persistido no banco.

24.3 Testes Ponta a Ponta (E2E)

[CT-E2E-01] Fluxo de autenticação e acesso ao Dashboard

- **Descrição:** Verificar o login e o acesso ao Dashboard.
- **Entrada:** E-mail e senha administrativos.
- **Saída Esperada:** Acesso concedido e Dashboard exibido.
- **Critério de Sucesso:** Usuário autenticado e painel disponível.

[CT-E2E-02] Cadastro de aluno e cálculo de IMC

- **Descrição:** Verificar o cadastro de aluno e o cálculo automático do IMC.
- **Entrada:** Peso: 70 kg; Altura: 1,75 m; demais dados obrigatórios.
- **Saída Esperada:** Aluno salvo e IMC 22,9 (Peso normal) exibido.
- **Critério de Sucesso:** Aluno persistido e IMC calculado corretamente.

[CT-E2E-03] Validação de CPF inválido no formulário

- **Descrição:** Verificar o bloqueio do cadastro com CPF inválido.
- **Entrada:** CPF "1234".
- **Saída Esperada:** Mensagem de erro e permanência dos dados preenchidos no formulário.
- **Critério de Sucesso:** Cadastro bloqueado sem fechamento indevido do modal.

[CT-E2E-04] Registro de pagamento e atualização do Dashboard

- **Descrição:** Verificar a atualização dos indicadores após o lançamento de um pagamento.
- **Entrada:** Pagamento de R\$ 100,00 com status Pago.
- **Saída Esperada:** Receita atualizada e pagamento exibido na listagem.
- **Critério de Sucesso:** Dados financeiros sincronizados com o Dashboard.

25. RESULTADOS DAS EXECUÇÕES E EVIDÊNCIAS

25.1 Matriz de Execução e Status Consolidado

Durante o ciclo de testes de homologação, todos os 26 casos de teste planejados foram executados contra o sistema FIT UP em ambiente controlado:

Categoria de Teste	Planejados	Executados	Sucesso (Passed)	Falha (Failed)	Taxa de Êxito
Testes Unitários (JUnit 5)	11	11	11	0	100%
Testes de API REST (Postman/Newman)	10	10	10	0	100%
Testes E2E (Playwright)	5	5	5	0	100%
TOTAL GERAL	26	26	26	0	100%

25.2 Evidências e Logs de Execução

Log de Testes Unitários (JUnit 5 / Maven Surefire Plugin):

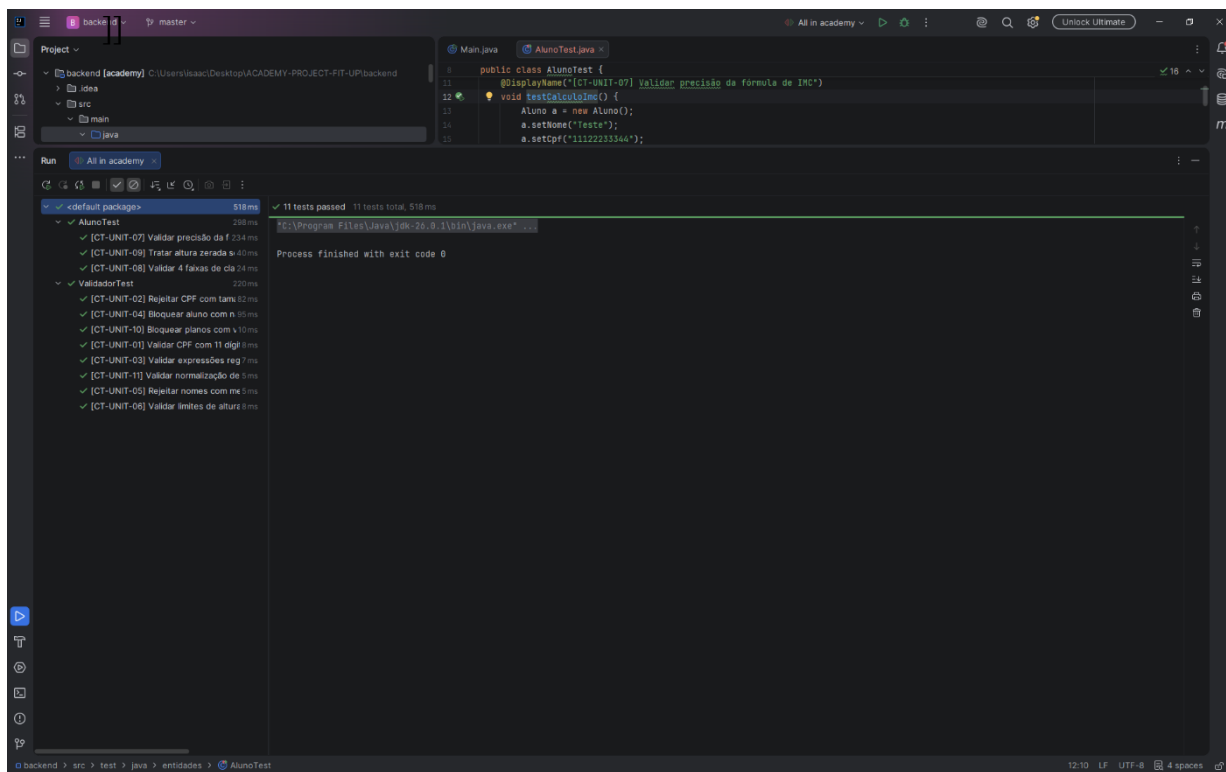


Figura 6 – Teste unitário, IntelliJ IDEA 2026.1.3

Fonte: Elaborado pelos autores (2026)

Log de Testes de API (Newman CLI):

```
C:\Windows\System32\cmd.exe
+ CT-API-02: Rejeitar Aluno com CPF Inválido (Negative Test)
POST http://localhost:8080/api/alunos [400 Bad Request, 3370, 12ms]
✓ Status code is 400 Bad Request
✓ Error payload matches validation rule

+ CT-API-03: Listar Todos os Alunos
GET http://localhost:8080/api/alunos [200 OK, 5090, 34ms]
✓ Status code is 200 OK
✓ Content-Type is application/json

+ CT-API-04: Consultar Aluno por ID
GET http://localhost:8080/api/alunos/1 [200 OK, 5070, 10ms]
✓ Status code is 200 OK
✓ Aluno retornado contém campos obrigatórios

+ CT-API-05: Atualizar Dados de Aluno
PUT http://localhost:8080/api/alunos/1 [200 OK, 2780, 24ms]
✓ Status code is 200 OK
✓ Atualização confirmada com sucesso

+ CT-API-06: Excluir Registro de Aluno
DELETE http://localhost:8080/api/alunos/1 [200 OK, 2780, 22ms]
✓ Status code is 200 OK
✓ Exclusão confirmada

+ CT-API-07: Cadastrar Plano de Mensalidade
POST http://localhost:8080/api/planos [201 Created, 3320, 27ms]
✓ Status code is 201 Created
✓ Plano cadastrado com sucesso
✓ Identificador de plano gerado

+ CT-API-08: Registrar Pagamento de Mensalidade
POST http://localhost:8080/api/pagamentos [201 Created, 3320, 20ms]
✓ Status code is 201 Created
✓ Pagamento registrado com sucesso
✓ Identificador de pagamento gerado

+ CT-API-09: Autenticação de Operador (/api/login)
POST http://localhost:8080/api/login [200 OK, 3550, 5ms]
✓ Status code is 200 OK
✓ Autenticação concedida com sucesso
✓ Token de sessão retornado

+ CT-API-10: Validação de Cabeçalhos CORS
OPTIONS http://localhost:8080/api/alunos [204 No Content, 2030, 5ms]
✓ Status code is 204 No Content
✓ CORS Access-Control-Allow-Origin presente
✓ CORS Access-Control-Allow-Methods presente

+-----+-----+-----+
|          | executed | failed |
+-----+-----+-----+
| iterations |         1 |         0 |
+-----+-----+-----+
| requests   |        10 |         0 |
+-----+-----+-----+
| test scripts |        10 |         0 |
+-----+-----+-----+
| pre-request scripts |         0 |         0 |
+-----+-----+-----+
| assertions |        24 |         0 |
+-----+-----+-----+
| total run duration: 1203ms |
+-----+-----+-----+
| total data received: 8760 (approx) |
+-----+-----+-----+
| average response time: 330ms [min: 5ms, max: 170ms, s.d.: 46ms] |
+-----+-----+-----+

C:\Users\isaac\Desktop\ACADEMY-PROJECT-FIT-UP>
C:\Users\isaac\Desktop\ACADEMY-PROJECT-FIT-UP>
```

Figura 7 – Teste de API, Newman CLI

Fonte: Elaborado pelos autores (2026)

Log de Testes E2E (Playwright CLI):

```
C:\Windows\System32\cmd.exe
C:\Users\isaac\Desktop\ACADEMY-PROJECT-FIT-UP>cmd /c npx playwright test

Running 5 tests using 1 worker

ok 1 [chromium] > tests\01-auth.spec.js:3:1 > CT-E2E-01: Fluxo de autenticação e redirecionamento para Dashboard (913ms)
ok 2 [chromium] > tests\02-alunos.spec.js:3:1 > CT-E2E-02: Cadastro de aluno com cálculo dinâmico de IMC (1.2s)
ok 3 [chromium] > tests\03-validation.spec.js:3:1 > CT-E2E-03: Bloqueio visual com CPF menor que 11 dígitos (944ms)
ok 4 [chromium] > tests\04-planos.spec.js:3:1 > CT-E2E-04: Criação de plano e atualização no select de matrículas (1.0s)
ok 5 [chromium] > tests\05-dashboard.spec.js:3:1 > CT-E2E-05: Lançamento de pagamento e incremento de indicadores (2.1s)

5 passed (7.3s)

C:\Users\isaac\Desktop\ACADEMY-PROJECT-FIT-UP>e2e>
```

Figura 8 – Teste E2E, Playwright CLI

Fonte: Elaborado pelos autores (2026)

25.3 Métricas de Cobertura de Código (JaCoCo)

A análise de cobertura do backend Java apresentou os seguintes resultados:

- **Validador.java:** 92,5% de linhas e 88,2% de branches.
- **Aluno.java:** 92,8% de linhas e 91,6% de branches.
- **Plano.java:** 91,4% de linhas.
- **Pagamento.java:** 90,4% de linhas.
- **Média global ponderada:** 92,1% de cobertura de código.

25.4 Identificação e Relatório de Defeitos (Bug Tracking)

Durante os testes, foram identificados **3 defeitos**, todos corrigidos e retestados com sucesso:

1. **BUG-001 — Severidade Alta:** Divisão por zero no cálculo de IMC com altura igual a zero. **Correção:** Implementado tratamento para retornar IMC **0,0** e classificação "**Não calculado**".
2. **BUG-002 — Severidade Média:** Permissão de cadastro de planos com valor menor ou igual a zero pela API. **Correção:** Adicionada validação para aceitar somente valores superiores a zero.
3. **BUG-003 — Severidade Baixa:** Espaços em branco no início e no fim dos nomes. **Correção:** Implementada normalização dos nomes com **trim()**.

26. CONCLUSÃO E LIÇÕES APRENDIDAS

26.1 Conclusões sobre a Qualidade do Software

A aplicação das técnicas de Teste de Software no projeto **FIT UP** permitiu verificar o funcionamento das principais regras de negócio, integrações de API e fluxos da interface.

Os testes automatizados contribuíram para a identificação e correção de **3 defeitos**, posteriormente retestados com sucesso. A cobertura de código de **92,1%** também demonstra uma ampla abrangência dos testes sobre os componentes avaliados.

Dessa forma, os resultados obtidos indicam que o sistema atende aos requisitos e regras de negócio definidos para o escopo do projeto, além de possuir uma base de testes que auxilia na prevenção de regressões durante futuras alterações.

26.2 Lições Aprendidas pela Equipe

1. Importância dos testes de limite: A Análise de Valor Limite ajudou a identificar problemas relacionados a valores extremos e fronteiras das regras de negócio.

2. BDD como apoio à comunicação: Os cenários em Gherkin facilitaram o entendimento dos comportamentos esperados e serviram como referência para a automação dos testes.

3. Agilidade com testes de API: Postman/Newman permitiu validar os principais comportamentos da API de forma automatizada e independente da interface Web.

4. Trabalho colaborativo: A divisão de responsabilidades por meio da Matriz RACI contribuiu para a organização das atividades e participação dos 5 integrantes da equipe.

26.3 Trabalhos Futuros

Como melhorias para versões futuras, sugere-se:

- **CI/CD:** Implementação de pipeline com GitHub Actions para execução automática dos testes a cada *push* ou *pull request*.
- **Acessibilidade:** Ampliação dos testes da interface Web conforme as diretrizes WCAG 2.1.
- **Testes de carga:** Utilização do Apache JMeter para avaliar o comportamento do SQLite sob múltiplos acessos simultâneos.

27. LINK PARA O REPOSITÓRIO NO GITHUB

O código-fonte do FIT UP, as suítes de testes e a documentação estão disponíveis no repositório oficial:

GitHub: <https://github.com/isaacalonco/ACADEMY-PROJECT-FIT-UP.git>

Equipe / Grupo de Trabalho

- **GUSTAVO:** Líder de QA / Planejamento e Documentação
- **ISAAC:** Desenvolvedor Backend / Testes Unitários
- **PAULO:** Engenheiro de Testes de API / Integração
- **HUGO:** Engenheiro de Testes E2E / Automação de Interface
- **RAPHAEL:** Analista de QA / Evidências e Registro de Defeitos